Kernel-Based Constrained Energy Minimization for Hyperspectral Mixed Pixel Classification

Kenneth Yeonkong Ma and Chein-I. Chang, *Life Fellow, IEEE*

***Abstract*— One fundamental task of hyperspectral imaging is spectral unmixing. In this case, the conventional pure pixel-based hyperspectral image classification (HSIC) may not work effectively for mixed pixels. This article presents a kernel-based approach to hyperspectral mixed pixel classification (HMPC) which includes two nonlinear mixed pixel classifiers, kernel constrained energy minimization (KCEM) and kernel linearly constrained minimum variance (KLCMV) to replace the widely used pure pixel-based support vector machine (SVM) classifier. Interestingly, what the binary-class and multiclass SVM classifiers are to pure pixel-based HSIC can be similarly derived for what a single-class KCEM detector and a multiclass KLCMV detector are to HMPC. In particular, the commonly used discrete classification map-based hard classification measures, average accuracy (AA) and overall accuracy (OA) for performance evaluation can be further generalized to real-valued mixed class abundance fractional map-based soft classification measures via 3-D receiver operating characteristic (3-D ROC) analysis-derived detection measures. Extensive experiments are conducted to demonstrate the utility of HMPC where KCEM/KLCMV not only significantly improve the classification performance of CEM/LCMV-based classifiers but also outperform many existing spectral-spatial classification methods.**

***Index Terms*— 3-D receiver operating characteristic (3-D ROC), constrained energy minimization (CEM), hyperspectral image classification (HSIC), hyperspectral mixed pixel classification (HMPC), kernel CEM (KCEM), kernel LCMV (KLCMV), linearly constrained minimum variance (LCMV).**

NOMENCLATURE

AA	Average accuracy.
AAUC	Average AUC.
Ascore	Average score.
AUC	Area under curve.
BKG	Background.
CEM	Constrained energy minimization.
HSIC	Hyperspectral image classification.
HMPC	Hyperspectral mixed pixel classification.
IID	Intrinsic image decomposition.
IRTS	Iterative random training sampling.
KCEM	Kernel CEM.
KLCMV	Kernel linearly constrained minimum variance.
LCMV	Linearly constrained minimum variance.
NKCEM	Nyström method-based KCEM.
NKLCMV	Nyström method-based KLCMV.
OA	Overall accuracy.
OAUC	Overall AUC.
Oscore	Overall score.
OTVCA	Orthogonal total variation component analysis.
PR	Precision rate.
ROC	Receiver operating characteristic.
SNPR	Signal-to-noise probability ratio.
SVM	Support vector machine.

Manuscript received March 18, 2021; revised April 25, 2021; accepted May 26, 2021. Date of publication June 25, 2021; date of current version January 12, 2022. The work of Chein-I. Chang was supported by the Fundamental Research Funds for Central Universities under Grant 3132019341. *(Corresponding author: Chein-I. Chang.)*

Kenneth Yeonkong Ma is with the Remote Sensing Signal and Image Processing Laboratory, Department of Computer Science and Electrical Engineering, University of Maryland, Baltimore County, Baltimore, MD 21250 USA (e-mail: kennethma1619@gmail.com).

Chein-I. Chang is with the Center for Hyperspectral Imaging in Remote Sensing (CHIRS), Information and Technology College, Dalian Maritime University, Dalian 116026, China, also with the Remote Sensing Signal and Image Processing Laboratory, Department of Computer Science and Electrical Engineering, University of Maryland, Baltimore County, Baltimore, MD 21250 USA, and also with the Department of Computer Science and Information Management, Providence University, Taichung City 02912, Taiwan (e-mail: cchang@umbc.edu).

Digital Object Identifier 10.1109/TGRS.2021.3085801

I. INTRODUCTION

HSIC has received considerable attention in the past. One widely used approach is spectral–spatial (SS) methods [1]–[20], which implement a pure pixel-based classifier, such as SVM [1] or multinomial logistic regression (MLR) [7] coupled with spatial filters such as Gabor filtering [2], extended morphological profile (EMP) [3], or edge preserving filtering (EPF) [4], [18], [20] to capture spatial contexture information. Most recently, fusion of SS classifiers was investigated for further improvement in [21]–[23]. Nevertheless, as a result of using hundreds of contiguous spectral bands, a hyperspectral data sample is generally a mixture of multiple known or unknown material signatures, which cannot be resolved by traditional pure pixel-based spectral classifiers. Specifically, such mixing activities generally occur with data samples mixed by BKG samples. On the contrary, HSIC has been performed using the provided ground truth (GT) by excluding the BKG samples from classification. Accordingly, BKG never becomes an issue because many may believe that BKG samples are not important or of no interest. This is misleading and not true. The fact that a data sample is considered as a BKG sample is simply because it cannot be labeled. Two causes contribute to BKG samples. One is that the data to be processed are too complicated. It may be very costly to provide a complete GT. Another is that a hyperspectral imaging sensor can uncover many subtle material substances, which cannot be identified by prior knowledge or visual inspection. Under such a circumstance it is impossible

1558-0644 © 2021 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

to label these unknown substances. In many applications, these substances are, indeed, very important such as anomalies. This also explains why anomaly detection has been a fundamental task in hyperspectral data exploitation. In addition, those BKG samples on labeled class boundaries may be further mixed with labeled data samples to create mixed pixels. Unfortunately, most reported works in the literature have overlooked this crucial BKG/mixed pixel issues by focusing only on labeled data samples, which can be identified by GT. In this case, the traditional measures, e.g., kappa coefficient, OA, and AA, are generally used to measure the efficacy of a classifier. So, a classifier is claimed to perform well as long as its kappa, AA, and OA values are high.

To address the BKG/mixed pixel issues described above, two approaches derived from a subpixel detection perspective have been investigated. One utilized a well-known subpixel target detector, called CEM developed in [24]–[26], to design a single-class classifier, iterative CEM (ICEM) in [27]. The other used the adaptive beamformer [28], [29] to extend ICEM to a multiclass spectral classifier, called iterative LCMV (ILCMV) in [30] and [31]. These two mixed pixel-based classifiers replaced the commonly used and pure pixel-based SVM classifier as spectral mixed pixel classifiers to perform HMPC. Specifically, a new classification measure, called PR, was introduced to address the BKG issue. As shown by experiments in [27], [30], and [31], ICEM and ILCMV improved PR at expense of slight degradation in OA and AA compared to some most widely used SS methods such as EPF-based methods [4]. This mainly results from their inability in dealing with nonlinear separability problem in classification. Compared to many SS methods which used nonlinear kernel functions such as SVM, ICEM/ILCMV are still linear classifiers. So, technically speaking, it is unfair to compare the linear classifiers, ICEM/ILCMV, to such nonlinear classifiers even though the former performs only slightly worse than the latter in terms of AA and OA but actually does better than the latter in PR. This leads to a potential improvement for ICEM/ILCMV if nonlinear kernel functions used by SVM can be also introduced in ICEM/ILCMV as KCEM and KLCMV.

This article takes up this issue and further develops a KCEM/KLCMV-based approach to HMPC. Specifically, what the binary-class and multiclass SVM classifiers are to pure pixel-based HSIC can be exactly derived for what a single-class KCEM detector and multitarget KLCMV are to HMPC. However, two issues need to be addressed.

The first and immediate obstacle is that CEM/LCMV require complete GT to calculate class means to be used as desired class signatures. Because SVM is a training sample-based classifier, to make KCEM/KLCMV competitive to SVM, we need to develop training sample-based KCEM/KLCMV, which also use training samples instead of complete GT to calculate class means in the same way that SVM uses training samples to train the classifier. Fortunately, this issue has been resolved recently by an approach, called IRTS-CEM in [32], which proposed an iterative training sample selection approach to allow ICEM to calculate each class mean by training samples.

A second obstacle is the kernel matrix used in KCEM/KLCMV. Unlike SVM, which only kernelizes on its support vectors, KCEM/KLCMV must kernelize on all data samples because of their use of sample correlation matrix. This results in very high computational complexity. Specifically, for a given hyperspectral image (HSI) data cube with N data samples, $\Omega_{\mathrm{HSI}} = \{\mathbf{r}_n\}_{n=1}^{N}$, KCEM requires storing and computing an $N \times N$ positive semidefinite kernel matrix, $\mathbf{K}_{N\times N}$. A straightforward application to KCEM does not work. To mitigate this dilemma, Nyström method [33]–[38] is used to perform low-rank approximation (LRA) on the kernel matrix so that the efficiency of standard KCEM can be largely improved, referred to as NKCEM. The idea of the Nyström method is to select a subset of m_N data samples, called landmark data samples, to approximately calculate kernel matrix so that the computation of an $N \times N$ kernel matrix can be relaxed to an $m_N \times N$ data samples matrix. Using the similar derivations of NKCEM, NKLCMV can be also derived similarly with more complicated operations by replacing vectors with matrices. Once both NKCEM and NKLCMV are in place, their IRTS versions, IRTS-NKCEM and IRTS-NKLCMV, are readily applicable.

Furthermore, by appealing for IRTS-CEM, KCEM/KLCMV can be implemented in conjunction with IRTS to derive IRTS-KCEM and IRTS-KLCMV. The idea of IRTS was originally developed in [20], which randomly selected a set of training samples independently through an iterative process iteration by iteration. For training samples selected in each iteration the classification maps produced by KCEM are processed by a set of Gaussian filters to capture spatial correlation among KCEM-classified data samples. Then, these Gaussian-filtered classification maps are fed back to expand the current image cube to create a new image cube to be used to reimplement KCEM over again for the next round of iteration. Accordingly, training samples selected at each iteration are further updated by including extra class spatial information. Eventually, IRTS-KCEM performs as well as KCEM using the complete GT.

Because NKCEM/NKLCMV are mixed pixel-based classifiers, they produce real-valued abundance fractional maps for each of classes for classification. In this case, discrete classification maps-based classification measures, kappa, AA, and OA are not directly applicable. A threshold method must be implemented to convert the real-valued class abundance fraction maps to binary class classification maps at the expense of spectral abundance information. To further resolve this issue, several new classification measures are also derived from the 3-D ROC curve-based detection measures developed in [39] and [40] as soft classification measures in [41] to evaluate mixed pixel classification performance according to the NKCEM/NKLCMV-generated real-valued abundance classification maps. With implementing IRTS-KCEM/KLCMV as mixed pixel-based classifiers coupled with 3-D ROC curve-derived soft classification measures, a kernel-based approach to HMPC can be therefore developed.

As a conclusion, we summarize the contributions of this article as follows:

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

1) Two nonlinear training samples-based mixed pixel-based classifiers, IRTS-KCEM/KLCMV, are derived for HMPC.
2) Because KCEM/KLCMV suffers from an exceeding computational problem in large storage memory, NKCEM/NKLCMV are further developed by including the Nyström method to perform LRA on the kernel matrix to resolve computational complexity issue in KCEM/KLCMV.
3) IRTS-NKCEM used for single-class mixed pixel classification is further extended to IRTS-NKLCMV as multiclass mixed pixel-based classifier for HMPC in the sense that NKLCMV can classify all M classes in one shot classification.
4) Because NKCEM/NKLCMV are mixed pixel-based classifiers, which produce real-valued classification maps, several new soft classification measures are also derived to evaluate the mixed pixel classification performance based on their generated abundance fractional maps for each of classes.
5) Last, and most importantly, a system can be designed to realize HMPC by combining NKCEM/NKLCMV, spatial filters, and IRTS, which has never been explored in the past.

II. Related Works

Classification can be considered as a special case of detection theory where a multiclass classification problem can be formulated as a multihypotheses detection problem with each class considered as a specific target under a separate hypothesis. Chang [42] was the first to develop a statistical detection theory approach to HSIC in the framework of multiple-target detection problem.

The idea of using a detector to perform HSIC was derived from the fact that a binary-class SVM classifier is indeed a detector. Because SVM uses a set of support vectors to find an optimal hyperplane to separate two classes of data samples, the hyperplane is actually determined by a threshold τ. By realizing this fact, Xue *et al.* [27] first developed a subpixel target detection approach to HSIC where CEM was used as a binary classifier to perform classification in a similar manner that a binary SVM classifier did. Then CEM was later extended to a multiple-target detector LCMV for multiclass classification in a similar way that a binary SVM classifier is extended to a multiclass SVM classifier.

Despite the fact that both SVM and CEM can be considered detectors, a key difference between them is that SVM is a hard classifier as opposed to CEM which is a soft classifier. In other words, SVM performs pure pixel classification by using discrete class membership assignments, whereas CEM performs mixed pixel classification by producing real-valued class abundance fractional maps. So, in order for CEM to perform classification, it requires a threshold value, τ, which is exactly the threshold τ used by SVM to find a hyperplane to separate data samples into two classes.

Because the CEM/LCMV are mixed pixel-based detectors, their generated class abundance fractional maps only provide spectral mixing information but not spatial information. To obtain spatial information, CEM/LCMV were extended to ICEM/ ILCMV in [27] and [30] by including a set of Gaussian filters to capture spatial information from the CEM/LCMV-detected class abundance fractional maps. These Gaussian-filtered class abundance fractional maps are then fed back to expand the current input data cube to form a new data cube using an iterative process.

However, CEM and LCMV are linear detectors and cannot solve nonlinear class inseparability problems. So, generally speaking, comparing ICEM/ILCMV to SVM-based SS methods for HSIC is not fair to ICEM/ILCMV because SVM is a kernel-based nonlinear classifier. Because of that, ICEM/ILCMV were shown to perform slightly worse than SVM-based SS methods in terms of AA and OA but better than PR in [27] and [30]. So, for ICEM/ILCMV to be competitive to SVM-based SS methods kernel functions must be introduced into CEM and LCMV. This leads to KCEM/KLCMV. Although KCEM was originally developed in [43], a direct use of KCEM in ICEM is not applicable due to excessive computational cost and enormous use of memory storage. To alleviate this problem, the Nyström method [33] is further used to avoid kernelizing the entire data samples.

Upon implementing ICEM/ILCMV it requires complete GT to calculate class means. In many practical applications, this may not be realistic when ICEM/ILCMV is compared to SVM, which only requires training samples. In order for ICEM/ILCMV to do the same as SVM does, ICEM was extended to IRTS-CEM in [32] using IRTS developed in [20].

As noted above, KCEM/KLCMV is basically detectors. When they are used as classifiers, they produced class abundance fractional maps rather than discrete classification maps. It is natural to take advantage of the 2-D ROC curve widely used in detection theory to evaluate the detection performance of a detector with an ROC curve plotted as a curve of detection probability P_{D} versus false alarm probability P_{F}. Then the AUC, denoted by $\mathrm{AUC}_{(\mathrm{D,F})}$, is calculated to measure the effectiveness of a detector. The higher value the $\mathrm{AUC}_{(\mathrm{D,F})}$ is, the better the detector is. Unfortunately, as demonstrated in [40] and [41] using such $\mathrm{AUC}_{(\mathrm{D,F})}$ for performance evaluation is not reliable. In other words, a detector can have a very high P_{D} value and also a very high P_{F} value because of the fact that P_{D} and P_{F} are calculated by the same threshold τ. Interestingly, the problem with $\mathrm{AUC}_{(\mathrm{D,F})}$ in target detection is similar to kappa/AA/OA used for classification because $\mathrm{AUC}_{(\mathrm{D,F})}$ and kappa/AA/OA do not take the BKG issue into account. To factor the BKG effect into the classification performance measure, several new classification measures from 3-D ROC analysis-based detection measures were derived in [41], referred to as soft classification measures as opposed to kappa/AA/OA as hard classification measures because the 3-D ROC curves are real-valued and classification maps are discrete.

III. CEM/LCMV

CEM was originally developed to detect targets at subpixel level. The idea arises from the linear adaptive beamforming [29] by identifying signals arriving from one specific

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

direction as desired signal source, $\mathbf{d}$, by designing a linear finite impulse response (FIR) filter to pass the desired signal while minimizing the output energy of signal sources from other undesired directions.

More specifically, let the original HSI data be Ω_{data}, where $\Omega_{\text{data}} = \{\mathbf{r}_n\}_{n=1}^{N}$ and $\mathbf{r}_n \in \Re^{L\times 1}$ is an nth data sample with L bands. Let the output of linear filter result of $\mathbf{r}_n$ be $y_n = \mathbf{w}^{\mathrm{T}}\mathbf{r}_n$. The averaged output energy of the entire image scene can be represented by the following equation:

$$\frac{1}{N}\sum_{i=1}^{N} y_n^2 = \mathbf{w}^{\mathrm{T}}\frac{1}{N}\left(\sum_{n=1}^{N}\mathbf{r}_n\mathbf{r}_n^{\mathrm{T}}\right)\mathbf{w} = \mathbf{w}^{\mathrm{T}}\mathbf{R}\mathbf{w} \tag{1}$$

where $\mathbf{R} \in \mathbb{R}^{L\times L}$ is the sample autocorrelation matrix and N is the total number of data samples. Then we can formulate the following constrained optimization problem:

$$\min_{\mathbf{w}}\left\{\mathbf{w}^{\mathrm{T}}\mathbf{R}\mathbf{w}\right\} \quad \text{subject to } \mathbf{d}^{\mathrm{T}}\mathbf{w} = 1 \tag{2}$$

to find an optimal $\mathbf{w}$ to minimize the averaged output energy in (2), while constraining signals from the desired direction of $\mathbf{d}$. The solution to (2) is given in [24] by

$$\mathbf{w}^{\mathrm{CEM}} = \mathbf{R}^{-1}\mathbf{d}\left(\mathbf{d}^{\mathrm{T}}\mathbf{R}^{-1}\mathbf{d}\right)^{-1}. \tag{3}$$

Different from CEM, which solves a single target detection problem, LCMV developed in [26] and [28] further extended CEM to detect multiple targets simultaneously by constraining a set of desired targets $\{\mathbf{d}_m\}_{m=1}^{M}$ specified by the desired target matrix, $\mathbf{D} = [\mathbf{d}_1\mathbf{d}_2\cdots\mathbf{d}_M]$. Then, a constrained optimization problem of LCMV can be reformulated as

$$\min_{\mathbf{w}}\left\{\mathbf{w}^{\mathrm{T}}\mathbf{R}\mathbf{w}\right\} \quad \text{subject to } \mathbf{D}^{\mathrm{T}}\mathbf{w} = \mathbf{1}_{M\times 1} \tag{4}$$

where $\mathbf{1}_{M\times 1} = (1_1, 1_2, \cdots, 1_M)^{\mathrm{T}}$ is an M-dimensional unity vector with each element in $\mathbf{1}_{M\times 1}$ used to constrain the direction of each desired signal. The optimal solution to (4) can be solved by

$$\mathbf{w}^{\mathrm{LCMV}} = \mathbf{R}^{-1}\mathbf{D}\left(\mathbf{D}^{\mathrm{T}}\mathbf{R}^{-1}\mathbf{D}\right)^{-1}\mathbf{1}_{M\times 1}. \tag{5}$$

To implement CEM/LCMV as a linear classifier, we first let $\{\boldsymbol{\mu}_m\}_{m=1}^{M}$ be the class sample means of M classes calculated by GT. A CEM classifier uses (3) to produce a real valued classification map for the mth class specified by

$$\delta_m^{\mathrm{CEM}}(\mathbf{r}_n) = \mathbf{r}_n^{\mathrm{T}}\mathbf{w}^{\mathrm{CEM}} = \mathbf{r}_n^{\mathrm{T}}\mathbf{R}^{-1}\boldsymbol{\mu}_m\left(\boldsymbol{\mu}_m^{\mathrm{T}}\mathbf{R}^{-1}\boldsymbol{\mu}_m\right)^{-1} \tag{6}$$

where $\delta_m^{\mathrm{CEM}}(\mathbf{r}_n)$ is the classification output of sample $\mathbf{r}_n$ in the mth class. However, for an M-classification problem, it requires CEM running M times using $\{\boldsymbol{\mu}_m\}_{m=1}^{M}$ to generate M real-valued classification maps. To obtain M real-valued classification maps simultaneously, the LCMV in (5) is generally by implementing all $\{\boldsymbol{\mu}_m\}_{m=1}^{M}$ as a class signature matrix, $\mathbf{D} = [\boldsymbol{\mu}_1\boldsymbol{\mu}_2\cdots\boldsymbol{\mu}_M]$, in which case the output of LCMV is given by

$$\delta^{\mathrm{LCMV}}(\mathbf{r}_n) = \mathbf{r}_n^{\mathrm{T}}\mathbf{W}^{\mathrm{LCMV}} = \mathbf{r}_n^{\mathrm{T}}\mathbf{R}^{-1}\mathbf{D}\left(\mathbf{D}^{\mathrm{T}}\mathbf{R}^{-1}\mathbf{D}^{\mathrm{T}}\right)^{-1}\mathbf{I}. \tag{7}$$

Note that the LCMV in (7) as a classifier is different than LCMV in (5) as a detector in the sense that the constrained unit vector $\mathbf{1}_{M\times 1}$ is replaced by $M \times M$ identity matrix, $\mathbf{I}$, so that the output of $\delta^{\mathrm{LCMV}}(\mathbf{r}_n)$ will be M-dimensional column vectors, each of which represents a real-valued classification map of one class.

Using CEM and LCMV, ICEM [27] and ILCMV [30], [31] can be developed for HMPC. Unlike traditional binary classifiers such as SVM generally produces "one-pixel-one-class" results, ICEM/ILCMV can be considered as a mixed-pixel classifier, which generates a real-value abundance fractional map for each class with each data sample represented by a mixture of multiple signature sources in $\mathbf{D}$. Comparing to many existing classification methods, the advantage of CEM/LCMV is to use the inverse of the sample correlation matrix $\mathbf{R}$ to suppress BKG, while constraining class sample means, $\{\boldsymbol{\mu}_m\}_{m=1}^{M}$, to perform an M-class classification. As a result, CEM and LCMV are easy to implement because they only require the knowledge of the class signatures to be specified. Because CEM/LCMV only use spectral information without taking interpixel spatial information into account, a set of Gaussian filters are further used to capture spatial information from their generated abundance fractional maps. The resulting spatial-filtered abundance fractional maps will be then fed back to the current image cube to create a new image cube to be sued for a next round of classification. The same process is repeated iteratively until a custom-designed stopping rule is met.

IV. KCEM/KLCMV

As a linear filter, CEM has its inability in dealing with nonlinear separability problem. To resolve this issue, a general approach is to use kernels to map data samples from the original space to a high-dimensional feature space. This section extends CEM and LCMV to their kernel-based counterparts, to be called KCEM and KLCMV.

Assume that $\mathbf{X}_{L\times N}$ is a data matrix formed by all data samples in Ω_{data} as $\mathbf{X}_{L\times N} = [\mathbf{r}_1\mathbf{r}_2\cdots\mathbf{r}_N]$, where each column vector represents a data sample. Let φ be a nonlinear function that maps an original data sample vector $\mathbf{r}_n$ into a high-dimensional Hilbert feature space, H using $\varphi : \Re^L \to H$ by $\mathbf{r}_n \to \varphi(\mathbf{r}_n)$ and $\mathbf{X} \to \mathbf{X}_\varphi$ with $\mathbf{X}_\varphi = [\varphi(\mathbf{r}_1)\varphi(\mathbf{r}_2)\cdots\varphi(\mathbf{r}_N)]$. The new autocorrelation matrix $\mathbf{R}_\varphi$ in the H-space can be computed and expressed as

$$\mathbf{R}_\varphi = \frac{1}{N}\sum_{n=1}^{N}\varphi(\mathbf{r}_n)\varphi^{\mathrm{T}}(\mathbf{r}_n) = \frac{1}{N}\mathbf{X}_\varphi\mathbf{X}_\varphi^{\mathrm{T}}. \tag{8}$$

Thus, operating CEM in the H-space yields

$$\delta^{\mathrm{CEM}}(\varphi(\mathbf{r}_n)) = \varphi(\mathbf{r}_n)^{\mathrm{T}}\mathbf{R}_\varphi^{-1}\left(\varphi(\mathbf{d})^{\mathrm{T}}\mathbf{R}_\varphi^{-1}\varphi(\mathbf{d})\right)^{-1}. \tag{9}$$

It should be noted that computing (9) is nearly impossible because of its high dimensionality. This leads to a major issue, which is how to use the kernel trick to operate (9). The strategy is very similar to kernel principal component analysis (KPCA) [44] and kernel Reed–Xiaoli detector (KRXD) [45] but could be more straightforward with no need of recentering all data samples in H.

Now, considering that λ_j is the jth eigenvalue of $\mathbf{R}_\varphi$ with its corresponding jth eigenvector, $\mathbf{v}_j$. Substituting (8) into

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

$\mathbf{R}_\varphi \mathbf{v}_j = \lambda_j \mathbf{v}_j$ yields

$$\mathbf{R}_\varphi \mathbf{v}_j = \lambda_j \mathbf{v}_j = \frac{1}{N}\sum_{n=1}^{N} \varphi(\mathbf{r}_n)\varphi(\mathbf{r}_n)^{\mathrm{T}}\mathbf{v}_j \tag{10}$$

which indicates that for any nonzero eigenvalues, λ_j, its corresponding eigenvector $\mathbf{v}_j$ must lie in the span of any $\varphi(\mathbf{r}_n)$. Because the term, $\varphi(\mathbf{r}_n)^{\mathrm{T}}\mathbf{v}_j$, is a scalar, we can further use a coefficient $\alpha_n^j = \varphi(\mathbf{r}_n)^{\mathrm{T}}\mathbf{v}_j / N\lambda_j$ to rewrite (10) as

$$\mathbf{v}_j = \frac{1}{N\lambda_j}\sum_{n=1}^{N} \varphi(\mathbf{r}_n)\varphi(\mathbf{r}_n)^{\mathrm{T}}\mathbf{v}_j = \sum_{n=1}^{N} \alpha_n^j \varphi(\mathbf{r}_n) = \mathbf{X}_\varphi \boldsymbol{\alpha}_j \tag{11}$$

where $\boldsymbol{\alpha}_j = (\alpha_1^j, \alpha_2^j, \cdots, \alpha_N^j)^{\mathrm{T}}$ is an N-dimensional column vector. Now multiplying both sides of (10) by $\mathbf{X}_\varphi$ gives rise to

$$\lambda_j \mathbf{X}_\varphi^{\mathrm{T}}\mathbf{v}_j = \mathbf{X}_\varphi^{\mathrm{T}}\mathbf{R}_\varphi \mathbf{v}_j. \tag{12}$$

Substituting (11) into (12) derives

$$\lambda_j \mathbf{X}_\varphi^{\mathrm{T}}\mathbf{X}_\varphi \boldsymbol{\alpha}_j = \mathbf{X}_\varphi^{\mathrm{T}}\frac{1}{N}\mathbf{X}_\varphi \mathbf{X}_\varphi^{\mathrm{T}}\mathbf{X}_\varphi \boldsymbol{\alpha}_j. \tag{13}$$

Define $K(\mathbf{r}_n, \mathbf{r}_j) = \varphi(\mathbf{r}_n)^{\mathrm{T}}\varphi(\mathbf{r}_j)$ and $\mathbf{K} = \mathbf{X}_\varphi^{\mathrm{T}}\mathbf{X}_\varphi \in \Re^{N\times N}$ to be a kernel matrix. Using the kernel trick, (13) we have

$$\lambda_j \mathbf{K}\alpha_j = \frac{1}{N}\mathbf{K}^2\boldsymbol{\alpha}_j \Rightarrow N\lambda_j \boldsymbol{\alpha}_j = \mathbf{K}\alpha_j. \tag{14}$$

From $\boldsymbol{\alpha}_j$ (14), turns out to be the eigenvector of $\mathbf{K}$ associated with eigenvalue $N\lambda_j$. Following the same derivation of KPCA in [36], $\boldsymbol{\alpha}_j$ can be normalized by $\sqrt{\lambda_j}$ to let $\tilde{\boldsymbol{\alpha}}_j = \boldsymbol{\alpha}_j\sqrt{\lambda_j}$ with $\|\tilde{\boldsymbol{\alpha}}_j\| = 1$. Then, $\mathbf{K}$ can be spectrally decomposed into

$$\mathbf{K} = N\tilde{A}\boldsymbol{\Lambda}\tilde{A}^{\mathrm{T}} \tag{15}$$

where $\tilde{\boldsymbol{A}} = [\tilde{\boldsymbol{\alpha}}_1\tilde{\boldsymbol{\alpha}}_2\cdots\tilde{\boldsymbol{\alpha}}_N]$ is an eigenvector matrix normalized by $\boldsymbol{\Lambda}^{(1/2)}$ with $\tilde{\boldsymbol{A}} = \mathbf{A}\Lambda^{(1/2)}$ and $\mathbf{A} = [\boldsymbol{\alpha}_1\boldsymbol{\alpha}_2\cdots\boldsymbol{\alpha}_N]$.

Finally, using (11) and (15), $\mathbf{R}_\varphi^{-1}$ can be derived as

$$\begin{aligned}\mathbf{R}_\varphi^{-1} &= \mathbf{V}\Lambda^{-1}\mathbf{V}^{\mathrm{T}} = \mathbf{X}_\varphi\mathbf{A}\Lambda^{-1}\left(\mathbf{X}_\varphi\mathbf{A}\right)^{\mathrm{T}} \\ &= \mathbf{X}_\varphi\mathbf{A}\Lambda^{(1/2)}\boldsymbol{\Lambda}^{-2}\left(\mathbf{X}_\varphi\mathbf{A}\Lambda^{(1/2)}\right)^{\mathrm{T}} \\ &= N^2\mathbf{X}_\varphi\tilde{\boldsymbol{A}}\frac{1}{N^2}\boldsymbol{\Lambda}^{-2}\tilde{\mathbf{A}}^{\mathrm{T}}\mathbf{X}_\varphi^{\mathrm{T}} = N^2\mathbf{X}_\varphi\mathbf{K}^{-2}\mathbf{X}_\varphi^{\mathrm{T}}.\end{aligned} \tag{16}$$

Now, if we substitute (16) into CEM in (9) and use the kernel function, we can derive KCEM as follows:

$$\begin{aligned}\delta^{\mathrm{KCEM}}(\mathbf{r}_n) &= \varphi(\mathbf{r}_n)^{\mathrm{T}}\mathbf{R}_\varphi^{-1}\varphi(\mathbf{d})\left(\varphi(\mathbf{d})^{\mathrm{T}}\mathbf{R}_\varphi^{-1}\varphi(\mathbf{d})\right)^{-1} \\ &= \varphi(\mathbf{r}_n)^{\mathrm{T}}\mathbf{X}_\varphi\mathbf{K}^{-2}\mathbf{X}_\varphi^{\mathrm{T}}\varphi(\mathbf{d})\left(\varphi(\mathbf{d})^{\mathrm{T}}\mathbf{X}_\varphi\mathbf{K}^{-2}\mathbf{X}_\varphi^{\mathrm{T}}\varphi(\mathbf{d})\right)^{-1} \\ &= \mathbf{k}_n^{\mathrm{T}}\mathbf{K}^{-2}\mathbf{k}_\mathbf{d}\left(\mathbf{k}_\mathbf{d}^{\mathrm{T}}\mathbf{K}^{-2}\mathbf{k}_\mathbf{d}\right)^{-1}\end{aligned} \tag{17}$$

where $\mathbf{k}_n = (K(\mathbf{r}_n, \mathbf{r}_1), K(\mathbf{r}_n, \mathbf{r}_2), \cdots, K(\mathbf{r}_n, \mathbf{r}_N))^{\mathrm{T}}$ is a column vector that contains the similarities of $\mathbf{r}_n$ and all data samples. In addition, $\mathbf{k}_\mathbf{d} = (K(\mathbf{d}, \mathbf{r}_1), K(\mathbf{d}, \mathbf{r}_2), \cdots, K(\mathbf{d}, \mathbf{r}_N))^{\mathrm{T}}$ includes all the kernels between $\mathbf{d}$ and $\mathbf{X}$. The kernel function that we use to compute similarity is RBF kernel [43], where $K(\mathbf{r}_n, \mathbf{r}_j) = \exp(-\|\mathbf{r}_n - \mathbf{r}_j\|^2/2\sigma^2)$. However, $\mathbf{K}$ may not be invertible. To deal with this issue, a common practice is to perform a singular value decomposition (SVD) on $\mathbf{K}$ and remove the eigenvalues that are too small compared with the largest eigenvalue. The details of the implementation of KCEM are as follows:

Algorithm 1 *Kernel Constrained Energy Minimization (KCEM)*

Input: Ω_{HSI}, $\mathbf{d}$, ε

1. Let $\Omega_{\mathrm{HSI}} = \{\mathbf{r}_n\}_{n=1}^N$ be an input HSI dataset and $\mathbf{d}$ the desired target signature
1) [2]. Create kernel matrix $\mathbf{K}$ using RBF kernel function
$$\mathbf{K} = [\mathbf{k}_1\mathbf{k}_2\ldots\mathbf{k}_N]$$
3. Kernelize desired target by letting
$$\mathbf{k}_\mathbf{d} = (K(\mathbf{d}, \mathbf{r}_1), K(\mathbf{d}, \mathbf{r}_2), \ldots, K(\mathbf{d}, \mathbf{r}_N))^{\mathrm{T}}$$
4. Perform SVD decomposition on $\mathbf{K}$ to let
$$\mathbf{K} = \mathbf{V}_\mathbf{K}\boldsymbol{\Lambda}_\mathbf{K}\mathbf{V}_\mathbf{K}^{\mathrm{T}} \tag{18}$$
5. Reduce the size of $\mathbf{V}_\mathbf{K}$ and $\mathbf{V}_\mathbf{K}$ to $\bar{\boldsymbol{V}}_\mathbf{K} = \mathbf{V}_\mathbf{K}(1:N, 1:t)$and $\bar{\boldsymbol{\Lambda}}_\mathbf{K} = \boldsymbol{\Lambda}_\mathbf{K}(1:t, 1:t)$ using a threshold ε to generate t
$$\mathbf{V}_\mathbf{K}(t+1, t+1) < \mathbf{V}_\mathbf{K}(1,1)\times\varepsilon \tag{19}$$
6. Using (17)–(19) to calculate KCEM
$$\delta^{\mathrm{KCEM}}(\mathbf{r}_n) = \frac{\mathbf{k}_n^{\mathrm{T}}\bar{\boldsymbol{V}}_\mathbf{K}\bar{\boldsymbol{\Lambda}}_\mathbf{K}^{-2}\bar{\boldsymbol{V}}_\mathbf{K}^{\mathrm{T}}\mathbf{k}_\mathbf{d}}{\mathbf{k}_\mathbf{d}^{\mathrm{T}}\bar{\boldsymbol{V}}_\mathbf{K}\bar{\boldsymbol{\Lambda}}_\mathbf{K}^{-2}\bar{\boldsymbol{V}}_\mathbf{K}^{\mathrm{T}}\mathbf{k}_\mathbf{d}} \tag{20}$$

Algorithm 1 can also be extended to KLCMV algorithm by replacing (20) with

$$\delta^{\mathrm{KLCMV}}(\mathbf{r}_n) = \frac{\mathbf{k}_n^{\mathrm{T}}\bar{\boldsymbol{V}}_\mathbf{K}\bar{\boldsymbol{\Lambda}}_\mathbf{K}^{-2}\bar{\boldsymbol{V}}_\mathbf{K}^{\mathrm{T}}\mathbf{K}_\mathbf{D}}{\mathbf{K}_\mathbf{D}^{\mathrm{T}}\bar{\boldsymbol{V}}_\mathbf{K}\bar{\boldsymbol{\Lambda}}_\mathbf{K}^{-2}\bar{\boldsymbol{V}}_\mathbf{K}^{\mathrm{T}}\mathbf{K}_\mathbf{D}}\mathbf{1}_{M\times 1} \tag{21}$$

where $\mathbf{K}_\mathbf{D} = [\mathbf{k}_{\mathbf{d}_1}\mathbf{k}_{\mathbf{d}_2}\cdots\mathbf{k}_{\mathbf{d}_M}]$ is a matrix that contains all similarities between all data samples and desired targets.

V. Nyström Method

One major issue in implementing KCEM/KLCMV is the requirement of its expensive computational times and memories resulting from manipulating the kernel matrix $\mathbf{K}_{N\times N}$. In general, there are three different types of nonlinear algorithms to compute kernels. The first type of algorithm only kernelize desired and undesired target knowledge such as kernel orthogonal subspace projection (KOSP) [46]. A second type only kernelizes training samples such as kernel Fisher's linear discriminant analysis (KFLDA) [47] or SVM [1]. A third one kernelizes entire dataset such as KPCA and KRXD. KCEM/KLCMV, indeed, belong to the third type of kernel algorithms because it requires to generate $\mathbf{K}_{N\times N}$ then calculate its inverse as shown in (17).

To relax the cost of storage and computation, one effective means is to perform LRA on $\mathbf{K}_{N\times N}$. That is, given a kernel matrix $\mathbf{K}_{N\times N}$ with rank m_N, which is much smaller than N ($m_N \ll N$), $\mathbf{K}_{N\times N}$ can be approximately decomposed into its low rank form expressed by

$$\mathbf{K}_{N\times N} \approx \mathbf{L}_{N\times m_N}\mathbf{L}_{N\times m_N}^{\mathrm{T}} \tag{22}$$

in which case dealing with $\mathbf{L}_{N\times m_N}$ would be much easier than manipulating $\mathbf{K}_{N\times N}$. Theoretically, SVD is the optimal LRA

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

method that spectrally decomposes $\mathbf{K}_{N\times N}$ into eigenvalue and eigenvector matrix then removes small eigenvalues to reduce the size of $\mathbf{V_K}$ and $\mathbf{\Lambda_K}$, as shown in (16). However, directly performing SVD on $\mathbf{K}_{N\times N}$ is very challenging and generally impractical because of its high computational complexity [37]. In the following, the Nyström method will be used to demonstrate how to perform LRA on $\mathbf{K}_{N\times N}$.

A. Nyström Method-Based KCEM Detector

Recently, the Nyström method [33]–[38] has shown to be one of most efficient LRA methods in terms of both computation and memory. The Nyström method was first developed by Williams and Seeger [33] and has been widely utilized to speed up many kernel-based algorithms such as manifold learning [35], KPCA [44], and KRXD [45] since then. The Nyström method is a sampling-based LRA method that chooses a subset of m_N data samples to form a so-called landmark data sample $\hat{\mathbf{X}}_{L\times m_N}$, and a kernel matrix can be approximated by Nyström method represented as

$$\mathbf{K}_{N\times N} \approx \mathbf{E}^{\mathrm{T}}_{m_N\times N}\hat{\mathbf{K}}^{-1}_{m_N\times m_N}\mathbf{E}_{m_N\times N} \tag{23}$$

where $\mathbf{E}_{m_N\times N} = K(\hat{\mathbf{X}}, \mathbf{X})$ is the matrix that contains all the similarities between all data samples in subset $\hat{\mathbf{X}}_{L\times m_N}$ and all data samples in $\mathbf{X}_{L\times N}$. Also, $\hat{\mathbf{K}}_{m_N\times m_N} = K(\hat{\mathbf{X}}, \hat{\mathbf{X}})$ is the kernel matrix that only includes the similarities of data samples in $\hat{\mathbf{X}}_{L\times m_N}$. Now, substituting (23) into KCEM detector in (17), we can obtain a Nyström-based KCEM (NKCEM) as follows:

$$\begin{aligned}\delta^{\mathrm{NKCEM}}(\mathbf{r}_n) &= \frac{\mathbf{k}_n^{\mathrm{T}}\mathbf{K}^{-2}\mathbf{k_d}}{\mathbf{k_d}^{\mathrm{T}}\mathbf{K}^{-2}\mathbf{k_d}} \\ &\approx \frac{\hat{\mathbf{K}}_n^{\mathrm{T}}\hat{\mathbf{K}}^{-1}\mathbf{E}\left(\mathbf{E}^{\mathrm{T}}\hat{\mathbf{K}}^{-1}\mathbf{E}\mathbf{E}^{\mathrm{T}}\hat{\mathbf{K}}^{-1}\mathbf{E}\right)^{-1}\mathbf{E}^{\mathrm{T}}\hat{\mathbf{K}}^{-1}\hat{\mathbf{K}}_{\mathbf{d}}}{\hat{\mathbf{K}}_{\mathbf{d}}^{\mathrm{T}}\hat{\mathbf{K}}^{-1}\mathbf{E}\left(\mathbf{E}^{\mathrm{T}}\hat{\mathbf{K}}^{-1}\mathbf{E}\mathbf{E}^{\mathrm{T}}\hat{\mathbf{K}}^{-1}\mathbf{E}\right)^{-1}\mathbf{E}^{\mathrm{T}}\hat{\mathbf{K}}^{-1}\hat{\mathbf{K}}_{\mathbf{d}}}\end{aligned} \tag{24}$$

where $\hat{\mathbf{K}}_n = K(\hat{\mathbf{X}}, \mathbf{r}_n)$ with $\hat{\mathbf{K}}_n \in \Re^{m_N}$ and $\mathbf{E}_{m_N\times N} = [\hat{\mathbf{k}}_1\hat{\mathbf{K}}_2\cdots\hat{\mathbf{K}}_N]$. Note that the inverse of the middle term in (24) can be computed by pseudo inverse. As a result, most of the terms get canceled and (24) can be further simplified as

$$\begin{aligned}\delta^{\mathrm{NKCEM}}(\mathbf{r}_n) &= \frac{\hat{\mathbf{K}}_n^{\mathrm{T}}\hat{\mathbf{K}}^{-1}\mathbf{E}\left(\hat{\mathbf{K}}^{-1}\mathbf{E}\right)^{\#}\left(\mathbf{E}\mathbf{E}^{\mathrm{T}}\right)^{\#}\left(\mathbf{E}^{\mathrm{T}}\hat{\mathbf{K}}^{-1}\right)^{\#}\mathbf{E}^{\mathrm{T}}\hat{\mathbf{K}}^{-1}\hat{\mathbf{K}}_{\mathbf{d}}}{\hat{\mathbf{K}}_{\mathbf{d}}^{\mathrm{T}}\hat{\mathbf{K}}^{-1}\mathbf{E}\left(\hat{\mathbf{K}}^{-1}\mathbf{E}\right)^{\#}\left(\mathbf{E}\mathbf{E}^{\mathrm{T}}\right)^{\#}\left(\mathbf{E}^{\mathrm{T}}\hat{\mathbf{K}}^{-1}\right)^{\#}\mathbf{E}^{\mathrm{T}}\hat{\mathbf{K}}^{-1}\hat{\mathbf{K}}_{\mathbf{d}}} \\ &= \frac{\hat{\mathbf{K}}_n^{\mathrm{T}}\left(\mathbf{E}\mathbf{E}^{\mathrm{T}}\right)^{\#}\hat{\mathbf{K}}_{\mathbf{d}}}{\hat{\mathbf{K}}_{\mathbf{d}}^{\mathrm{T}}\left(\mathbf{E}\mathbf{E}^{\mathrm{T}}\right)^{\#}\hat{\mathbf{K}}_{\mathbf{d}}}.\end{aligned} \tag{25}$$

As we can see from (25), the inverse of $\mathbf{K}_{N\times N}$ in (17) can indeed be simplified by finding the inverse of an $m_N \times m_N$ matrix (i.e., $\mathbf{E}_{m_N\times N}\mathbf{E}^{\mathrm{T}}_{m_N\times N}$) in (25). If $\mathbf{E}\mathbf{E}^{\mathrm{T}}$ is invertible then $(\mathbf{E}\mathbf{E}^{\mathrm{T}})^{\#} = (\mathbf{E}\mathbf{E}^{\mathrm{T}})^{-1}$.

The next important issue is how to select appropriate landmark data points to perform LRA on $\mathbf{K}_{N\times N}$. One popular sampling scheme used by the Nyström method is random sampling, which randomly chooses a subset of data samples from $\mathbf{X}_{L\times N}$. Here, we adopt the clustered Nyström method [33], which utilizes the k-means clustering to find m_N clustering centers to be used as landmark data points. One advantage of using k-means is that it can achieve better lower approximation error than other sampling schemes [33]. Another is that it can reduce randomness resulting from random sampling. Nevertheless, any other types of sampling methods could also be used to replace k-means. The step-by-step implementations of NKCEM are described in the following algorithm, Algorithm 2, which describes step-by-step implementation of NKCEM:

Algorithm 2 *Clustered Nyström Method-based KCEM*

Input: Ω_{HSI}, m_N, $\mathbf{d}$

1. Let $\Omega_{\mathrm{HSI}} = \{\mathbf{r}_n\}_{n=1}^N$ and $\mathbf{X} = [\mathbf{r}_1\mathbf{r}_2\ldots\mathbf{r}_N]$
2. Using k-means clustering to find m_N cluster centers as landmark data points
$$\hat{\mathbf{X}} = k\mathrm{means}(\mathbf{X}, m_N)$$
3. Let $\mathbf{E} = [\hat{\mathbf{K}}_1\hat{\mathbf{K}}_2\ldots\hat{\mathbf{K}}_N]$ where $\hat{\mathbf{K}}_n = K(\hat{\mathbf{X}}, \mathbf{r}_n)$
4. Perform SVD decomposition on $\mathbf{E}\mathbf{E}^{\mathrm{T}}$ to let
$$\mathbf{E}\mathbf{E}^{\mathrm{T}} = \mathbf{V_E}\mathbf{\Lambda_E}\mathbf{V_E}^{\mathrm{T}}$$
5. Reduce the size of $\mathbf{V_E}$ and $\mathbf{\Lambda_E}$ to $\bar{\mathbf{V}}_{\mathbf{E}} = \mathbf{V_E}(1:m_N, 1:t)$ and $\bar{\mathbf{\Lambda}}_{\mathbf{E}} = \mathbf{\Lambda_E}(1:t, 1:t)$ by using a threshold ε
$$\mathbf{\Lambda_E}(t+1, t+1) < \mathbf{\Lambda_E}(1,1)\times\varepsilon \tag{26}$$
6. Using (25) and (26) to generate NKCEM detection result
$$\delta^{\mathrm{NKCMV}}(\mathbf{r}_n) = \frac{\hat{\mathbf{K}}_n^{\mathrm{T}}\bar{\mathbf{V}}_{\mathbf{E}}\bar{\mathbf{\Lambda}}_{\mathbf{E}}^{-1}\bar{\mathbf{V}}_{\mathbf{E}}^{\mathrm{T}}\hat{\mathbf{k}}_{\mathbf{d}}}{\hat{\mathbf{K}}_{\mathbf{d}}^{\mathrm{T}}\bar{\mathbf{V}}_{\mathbf{E}}\bar{\mathbf{\Lambda}}_{\mathbf{E}}^{-1}\bar{\mathbf{V}}_{\mathbf{E}}^{\mathrm{T}}\hat{\mathbf{k}}_{\mathbf{d}}} \tag{27}$$

Following the same idea of deriving NKCEM, we can also derive NKLCMV as follows:

$$\delta^{\mathrm{NKLCMV}}(\mathbf{r}_n) = \frac{\hat{\mathbf{K}}_n^{\mathrm{T}}\bar{\mathbf{V}}_{\mathbf{E}}\bar{\mathbf{\Lambda}}_{\mathbf{E}}^{-1}\bar{\mathbf{V}}_{\mathbf{E}}^{\mathrm{T}}\hat{\mathbf{K}}_{\mathbf{D}}}{\hat{\mathbf{K}}_{\mathbf{D}}^{\mathrm{T}}\bar{\mathbf{V}}_{\mathbf{E}}\bar{\mathbf{\Lambda}}_{\mathbf{E}}^{-1}\bar{\mathbf{V}}_{\mathbf{E}}^{\mathrm{T}}\hat{\mathbf{K}}_{\mathbf{D}}} \tag{28}$$

where $\hat{\mathbf{K}}_{\mathbf{D}} = [\hat{\mathbf{K}}_{\mathbf{d}_1}\hat{\mathbf{k}}_{\mathbf{d}_2}\cdots\hat{\mathbf{K}}_{\mathbf{d}_M}]$.

B. Hyperspectral Image Classification Using NKCEM and NKLCMV

While implementing KCEM and KLCMV for HMPC, we need to calculate the mean of each class as the desired class signature by

$$\hat{\boldsymbol{\mu}}_m^{\varphi} = \frac{1}{|S_m^{\mathrm{training}}|}\sum_{n\in S_m^{\mathrm{training}}}\varphi(\mathbf{r}_n) \tag{29}$$

where $\hat{\boldsymbol{\mu}}_m^{\varphi}$ indicates mth class signatures in H. However, (29) cannot be explicitly computed. Fortunately, we can use a kernel to simply compute

$$\mathbf{k}_{\boldsymbol{\mu}_m} = \frac{1}{|S_m^{\mathrm{training}}|}\sum_{n\in S_m^{\mathrm{training}}}\mathbf{k}_n \tag{30}$$

where the KCEM classifier for mth class can be described by

$$\delta_m^{\mathrm{KCEM}}(\mathbf{r}_n) = \frac{\mathbf{k}_n^{\mathrm{T}}\mathbf{K}^{-2}\mathbf{k}_{\boldsymbol{\mu}_m}}{\mathbf{k}_{\boldsymbol{\mu}_m}^{\mathrm{T}}\mathbf{K}^{-2}\mathbf{k}_{\boldsymbol{\mu}_m}}. \tag{31}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

Similarly, the KLCMV classifier could also be generated by replacing $\mathbf{k}_{\boldsymbol{\mu}_m}$ with $\mathbf{K}_{\mathbf{D}_M}$ as

$$\delta^{\mathrm{KLCMV}}(\mathbf{r}_n) = \frac{\mathbf{k}_n^{\mathrm{T}}\mathbf{K}^{-2}\mathbf{K}_{\mathbf{D}_M}}{\mathbf{K}_{\mathbf{D}_M}^{\mathrm{T}}\mathbf{K}^{-2}\mathbf{K}_{\mathbf{D}_M}}\mathbf{I}_{M\times M} \tag{32}$$

where $\mathbf{K}_{\mathbf{D}_M} = [\mathbf{k}_{\boldsymbol{\mu}_1}\mathbf{k}_{\mu_2}\cdots\mathbf{k}_{\boldsymbol{\mu}_M}]$ is a matrix containing all $\mathbf{k}_{\boldsymbol{\mu}_m}$. Finally, we can use (25) and (28) to simplify (31) and (32), which result in

$$\delta_m^{\mathrm{NKCEM}}(\mathbf{r}_n) = \frac{\hat{\mathbf{K}}_n^{\mathrm{T}}\left(\mathbf{E}\mathbf{E}^{\mathrm{T}}\right)^{\#}\hat{\mathbf{K}}_{\boldsymbol{\mu}_m}}{\hat{\mathbf{K}}_{\boldsymbol{\mu}_m}^{\mathrm{T}}\left(\mathbf{E}\mathbf{E}^{\mathrm{T}}\right)^{\#}\hat{\mathbf{K}}_{\boldsymbol{\mu}_m}} \tag{33}$$

and

$$\delta^{\mathrm{NKLCMV}}(\mathbf{r}_n) = \frac{\hat{\mathbf{K}}_n^{\mathrm{T}}\left(\mathbf{E}\mathbf{E}^{\mathrm{T}}\right)^{\#}\hat{\mathbf{K}}_{\mathbf{D}_M}}{\hat{\mathbf{K}}_{\mathbf{D}_M}^{\mathrm{T}}\left(\mathbf{E}\mathbf{E}^{\mathrm{T}}\right)^{\#}\hat{\mathbf{K}}_{\mathbf{D}_M}}\mathbf{I}_{M\times M}. \tag{34}$$

Although KCEM and kernel target-constrained interference-minimized filter (KTCIMF) are not new and have been used for target detection [43], [49], respectively, to our best knowledge, the use of KCEM/KLCMV for HMPC has not been explored.

VI. IRTS-NKCEM/NKLCMV for HMPC

Although ICEM [27] and ILCMV [30], [31] have been developed as SS classifiers for HMPC, both CEM and LCMV require the complete GT to calculate class sample means to $\{\boldsymbol{\mu}_m\}_{m=1}^M$. In practical applications, this may not be realistic. Recently, an IRTS-CEM was developed in [32] to resolve this issue for ICEM so that CEM can still work by using limited training samples to calculate class means.

The idea of IRTS was originally developed in [20] to address the randomness resulting from different sets of training samples. IRTS made use of iterative feedback loops to expand image cubes in an iterative manner by adding to the current image cube spatial filtered classification maps produced by a different set of training samples in each iteration. Because more SS information of each data sample was fed back to the data cube, the classification uncertainty is then largely reduced and the misclassification rate is also decreased. In [20], IRTS was implemented as training sample selection for SS to improve the classification results over traditional SS classification. This section includes IRTS in NKCEM/NKLCMV to develop two new kernel-based SS classifiers, referred to as IRTS-NKCEM/NKLCMV.

Let the processing data cube at the kth iteration be denoted by $\boldsymbol{\Omega}^{(k)} = \{\mathbf{r}_n\}_{n=1}^N$. We use NKCEM in (25) or NKLCMV in (28) to produce spectral-based classification maps, $\mathrm{Smap}^{(k)} = \{\mathrm{Smap}_m^{(k)}\}_{m=1}^M$, which are further used as inputs to a set of Gaussian filters to generate Gaussian filtered classification maps, $\mathrm{SFmap}^{(k)} = \{\mathrm{SFmap}_m^{(k)}\}_{m=1}^M$. Then, the filtered classification map of mth class at the $(k-1)$st iteration, $\mathrm{SFmap}_m^{(k-1)}$ is fused with the mth classification map at the kth iteration $\mathrm{SFmap}_m^{(k)}$ to produce a fused result for each data sample $\mathbf{r}_n$ by taking their maximum value

$$\mathrm{MAXmap}_m^{(k)}(\mathbf{r}_n) = \max\left\{\mathrm{SFmap}_m^{(k)}(\mathbf{r}_n), \mathrm{SFmap}_m^{(k-1)}(\mathbf{r}_n)\right\}. \tag{35}$$

Because the training samples selected in the in the $(k-1)$st iteration and kth iteration are completely independent, the maximization process in (35) is able to jointly consider the information provided by two consecutive iterations. After fusion softmax function is used to transform $\{\mathrm{MAXmap}_m^{(k)}\}_{m=1}^M$ into *a posteriori* probability maps as

$$\mathrm{Pmap}_m^{(k)}(\mathbf{r}_n) = \frac{\exp\left(\beta\mathrm{MAXmap}_m^{(k)}(\mathbf{r}_n)\right)}{\sum_{j=1}^M \exp\left(\beta\mathrm{MAXmap}_j^{(k)}(\mathbf{r}_n)\right)} \tag{36}$$

where β is a parameter used to control the degree of concentration. The resulting probability maps would then be augmented into currently being processed image cube to form an expanded image cube by

$$\boldsymbol{\Omega}^{(k+1)} = \boldsymbol{\Omega}^{(k)} \cup \left\{\mathrm{Pmap}_m^{(k)}\right\}_{m=1}^M \tag{37}$$

which will be used as a new data cube to perform classification for the next iteration. The iterative process is repeated over and over again until $\{\mathrm{Bmap}_m^{(k)}\}_{m=1}^M$ satisfies the following stopping rule, which is Tanimoto index (TI) [49] described as follows:

$$\mathrm{TI}^{(k)} = \sum_{m=1}^M p(C_m)\frac{|\mathrm{Bmap}_m^{(k-1)} \cap \mathrm{Bmap}_m^{(k)}|}{|\mathrm{Bmap}_m^{(k-1)} \cup \mathrm{Bmap}_m^{(k)}|} \tag{38}$$

where $p(C_m)$ is the probability of the mth class C_m with $p(C_m) = (n_m/N)$ if the sample ratio is used, n_m is the number of data samples in C_m, and $\{\mathrm{Bmap}_m^{(k)}\}_{m=1}^M$ is the binary classification results produced by maximum a *posterior* (MAP), represented as

$$\mathrm{Bmap}_m^{(k)}(\mathbf{r}_n) = \begin{cases} 1; & \mathrm{MAXmap}_m^{(k)}(\mathbf{r}_n) \\ & > \mathrm{MAXmap}_{i\in\{1,\ldots,M\}, i\neq m}^{(k)}(\mathbf{r}_n) \\ 0; & \text{otherwise.} \end{cases} \tag{39}$$

The TI in (38) is a similarity measurement of classification maps between two consecutive iterations. The detailed implementation of IRTS-NKCEM/NKLCMV is described by in Algorithm 3 in the following with its graphic block diagram shown in Fig. 1.

A final note is noteworthy. Although IRTS-CEM has been used for HPMC [32], the IRTS introduced in this section has two major differences from IRTS-CEM. The first one is that IRTS-CEM did not include a fusion process in (35). The second one is that IRTS-CEM did not use the softmax function to normalize spatial filtered maps into probability maps before feedback process in (37). There are two criteria used to terminate IRTS-NKCEM. One is the traditional classification measures, OA/AA/PR, to evaluate the binary-classification results on $\{\mathrm{Bmap}_m^{(k)}\}_{m=1}^M$. The other is the 3-D-ROC analysis developed in [39]–[41] to measure the real-valued classification results in $\{\mathrm{MAXmap}_m^{(k)}\}_{m=1}^M$. These two criteria are defined in Sections VII and VIII.

VII. Classification Measures

This section summarizes the classification measures [41], [42] to be used for experiments as follows:

1) M = the number of classes.
2) n_{ij} = the number of samples in the jth class to be classified into the ith class.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

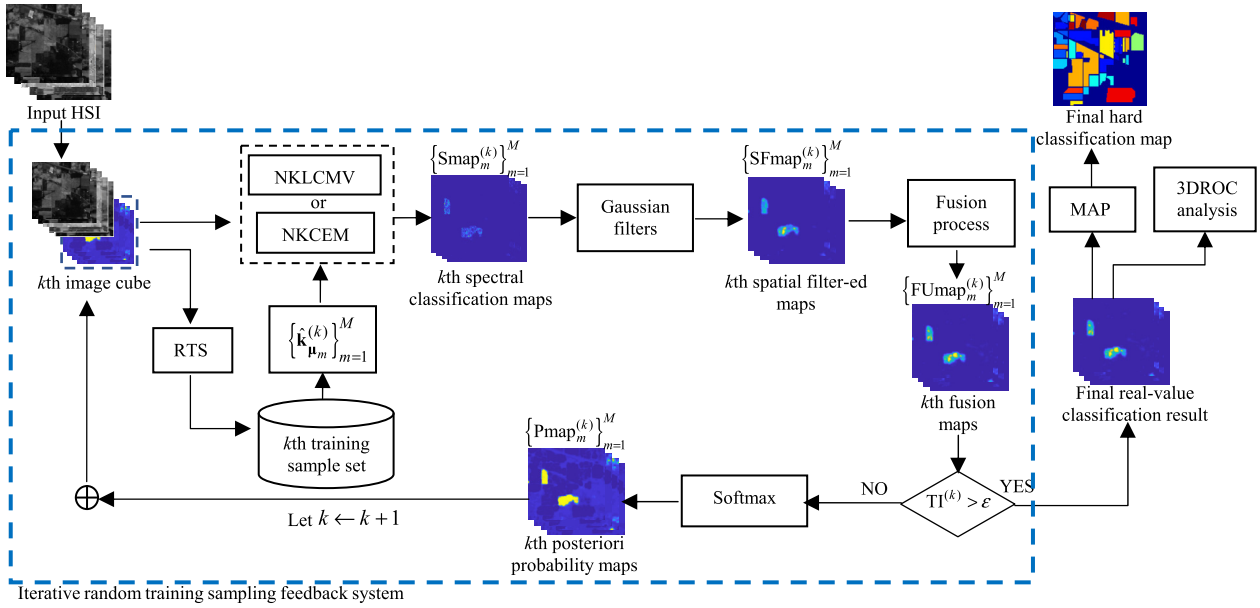


Fig. 1. Graphic diagram of the proposed IRTS-NKCEM and IRTS-NKLCMV.

Algorithm 3 IRTS-NKCEM/NKLCMV

Initial conditions: Ω_{HSI}, ε

1. Let $\Omega^{(0)} = \Omega_{\text{HSI}}$ be the original image cube. Set $k = 0$
2. At the kth iteration, randomly select training samples for each of classes, $S_{\text{training}} = \{S_m^{(k)}\}_{m=1}^M$ from $\Omega^{(k)}$, and calculate $\{\hat{\mathbf{k}}_{\boldsymbol{\mu}_m}\}_{m=1}^M$ using (30)
3. Use NKCEM or NKLCMV to produce kth soft-spectral classification map, $\{\text{Smap}_m^{(k)}\}_{m=1}^M$
4. Utilize a set of Gaussian filters on $\{\text{Smap}_m^{(k)}\}_{m=1}^M$ to produced SF-ed maps, $\{\text{SFmap}_m^{(k)}\}_{m=1}^M$
5. If $k = 0$, set $k \leftarrow k+1$, go to step 2. Otherwise, continue
6. Implement fusion process in (34) to produce kth fusion maps, $\{\text{MAXmap}_m^{(k)}\}_{m=1}^M$
7. Use MAP to $\{\text{MAXmap}_m^{(k)}\}_{m=1}^M$ to produce binary maps, $\{\text{Bmap}_m^{(k)}\}_{m=1}^M$
8. Check if $\text{TI}^{(k)} > \varepsilon$. If yes, the algorithm terminate. Otherwise, set $k \leftarrow k+1$ and go to next step
9. Produce $\{\text{Pmap}_m^{(k)}\}_{m=1}^M$ using (36) and feed back probability maps to form a new image cube using (37) and go to step 2

Algorithm output: $\{\text{Bmap}_m^{(k)}\}_{m=1}^M$ and $\{\text{MAXmap}_m^{(k)}\}_{m=1}^M$

3) n_{jj} = the number of samples in the jth class correctly classified into the jth class.
4) n_j = the number of data samples in the jth class, i.e., $n_j = \sum_{i=1}^M n_{ij}$.
5) $\hat{n}_{ij}$ = the number of data samples classified in the ith class, which are supposed to in the jth class, C_j.
6) $\hat{n}_{ij} = n_{ij}$ with n_{ij} interpreted from *a priori* point of view and $\hat{n}_{ij}$ interpreted from *a posteriori* point of view.
7) $\hat{n}_i$ = the number of data samples classified in the ith class, i.e., $\hat{n}_i = \sum_{j=1}^M \hat{n}_{ij}$.
8) $\tilde{n}_j = \sum_{i=1, i\neq j}^M n_{ij}$ = the number of samples in the ith class with $i \neq j$ falsely classified into the jth class.
9) N = the total number of data samples, $N = \sum_{j=1}^M n_j$.
10) C_j = the set of samples in the jth class.
11) $\tilde{C}_j$ = the set of data samples from classes $\{C_i\}_{i=1,i\neq j}^M$ classified into C_j.
12) $p(C_j) = (n_j/N)$.

In the following, we define several classification measures:

1) Class accuracy

$$P_{\text{A}}(C_j) = \text{accuracy of the } j \text{ th class} = P(C_{jj}|C_j) = \frac{n_{jj}}{n_j}. \tag{40}$$

2) Average PR

$$P_{\text{APR}} = \frac{1}{M}\sum_{i=1}^M p_{\text{PR}}(\hat{C}_i) = \frac{1}{M}\sum_{i=1}^M \frac{\hat{n}_{ii}}{\hat{n}_i}. \tag{41}$$

3) PR

$$P_{\text{PR}} = \sum_{i=1}^M \frac{\hat{n}_i}{\hat{N}} p_{\text{PR}}(\hat{C}_i). \tag{42}$$

4) OA

$$P_{\text{OA}} = p_{\text{overall-accuracy}}\left(\{C_j\}_{j=1}^M\right) = \sum_{j=1}^M p(\text{C}_j)\, p(\text{C}_{jj}|\text{C}_j) = \frac{1}{N}\sum_{j=1}^M n_j \left(\frac{n_{jj}}{\sum_{i=1}^M n_{ij}}\right). \tag{43}$$

5) AA

$$P_{\text{AA}} = \frac{1}{M}\sum_{j=1}^M P_{\text{A}}(C_j) = \frac{1}{M}\sum_{i=1}^M \frac{n_{ii}}{n_i}. \tag{44}$$

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

VIII. 3-D ROC-Derived Soft Classification Measures

In this section, we present several new 3-D ROC curve-based soft classification measures derived from [41] and [42] to evaluate the real-valued classification performance resulting from the mixed pixel classifiers. Generally, ROC analysis is not applicable to HMPC because the conventional classification approach only produces discrete classification maps where common measures to evaluate performance are OA/AA/kappa. On the contrary, because IRTS-NKCEM/NKLCMV are detection techniques that produce real-valued classification maps, the ROC analysis can be used to evaluate their classification performance in terms of both classification accuracy and BKG suppressibility where the target detection probability, the false alarm probability P_F, and the noise can be interpreted as class accuracy P_A in (40) misclassification error and BKG, respectively.

In the traditional 2-D ROC analysis, only detection rate, P_D (i.e., P_A), and false alarm rate, P_F, are used to plot a curve of (P_D, P_F) to calculate $AUC_{(D,F)}$ for detection performance evaluation. However, a main issue arising in $AUC_{(D,F)}$ is that it can only evaluate the effectiveness of a detector but not detection performance because P_D and P_F can both be determined by the same threshold value τ. To address this issue, the 2-D ROC curves were extended to 3-D ROC of (P_D, P_F, τ) in [40] to calculate P_D and P_F separately by varying the values of τ. Using a 3-D ROC curve, three 2-D ROC curves, 2-D ROC curve of (P_D, P_F), 2-D ROC curve of (P_D, τ), and 2-D ROC curve of (P_F, τ) can be then generated. Furthermore, the AUC of the three 2-D ROC curves, $AUC_{(D,F)}$, $AUC_{(D,\tau)}$, and $AUC_{(F,\tau)}$, can be even further used as a base to derive various quantitative indicators to evaluate the real-valued classification results of each class produced by mixed pixel classifiers [41]. Specifically, the 2-D ROC curve of (P_F, τ) can be used to evaluate the BKG suppressibility and $AUC_{(D,\tau)}$ can be used to evaluate class accuracy, P_A. Moreover, we particularly include a measure called SNPR developed in [40] to jointly consider target detectability and BKG suppressibility defined by AUC_{SNPR}

$$0 \le \mathrm{AUC}_{\mathrm{SNPR}} = \frac{\mathrm{AUC}_{(\mathrm{D},\tau)}}{\mathrm{AUC}_{(\mathrm{F},\tau)}} \tag{45}$$

where the false alarm probability P_F can be assumed to be caused by BKG.

To generate an OAUC measure over M classes, the AAUC values over M class can further be calculated by two different ways in a similar manner that OA and AA are calculated. One is by class sample ratio, referred to as OAUC

$$\mathrm{OAUC} = \sum_{j=1}^{M} \frac{n_j}{N} \mathrm{AUC}_j. \tag{46}$$

Another is to assume that each class is equally significant, denoted by AAUC

$$\mathrm{AAUC} = \sum_{j=1}^{M} \frac{1}{M} \mathrm{AUC}_j. \tag{47}$$

Finally, the overall classification score developed in [41] is to further measure the overall performance of a classifier. The first one called Oscore, defined by

$$\mathrm{Oscore} = P_{\mathrm{OA}} + P_{\mathrm{PR}} + \mathrm{OAUC}_{(\mathrm{D,F})} + \mathrm{OAUC}_{(\mathrm{D},\tau)} - \mathrm{OAUC}_{(\mathrm{F},\tau)}. \tag{48}$$

A second one is Ascore defined by

$$\mathrm{Ascore} = P_{\mathrm{AA}} + P_{\mathrm{APR}} + \mathrm{AAUC}_{(\mathrm{D,F})} + \mathrm{AAUC}_{(\mathrm{D},\tau)} - \mathrm{AAUC}_{(\mathrm{F},\tau)}. \tag{49}$$

So, a higher value of P_{OA}, P_{AA}, P_{PR}, P_{APR}, $AUC_{(D,F)}$, and $AUC_{(D,\tau)}$ indicates a better classification performance. By contrast, a lower value of $AUC_{(F,\tau)}$ indicates a better BKG suppression ability and thus a better classification performance.

It should be noted that OAUC in (46), AAUC in (47), Oscorein (48), and Ascore in (49) are new and were not derived in [41].

IX. Real HSI Datasets Used for Experiments

To demonstrate the performance of proposed algorithms, the following three popular HSIs were used for experiments:

A. Purdue Indiana Indian Pines

The first one is the Purdue Indiana Indian Pines test site with an aerial view shown in Fig. 2(a) along with its GT of 17 class maps in Fig. 2(b) and their class labels in Fig. 2(c). Table I summarizes the number of data samples in each class. It is an airborne visible/infrared imaging spectrometer (AVIRIS) image scene and has size of $145 \times 145 \times 220$ pixel vectors with water-absorption bands (bands 104–108 and 150–163, 220). It should be noted that in this article, 200 bands were used by excluding 20 of water-absorption bands.

B. Salinas

A second image dataset is Salinas in Fig. 3(a), which is also an AVIRIS scene. It was collected over Salinas Valley, CA, USA, with a spatial resolution of 3.7-m per pixel with spectral resolution of 10 nm. It has size of $512 \times 217 \times 224$ with 20 water-absorption bands, 108–112, 154–167, and 224. So, a total of 224 bands were used for experiments. Fig. 3(b) and (c) shows the color composite of the Salinas image along with the corresponding GT class labels. Unlike the Purdue data Salinas scene does not have an in issue in imbalanced classes as summarized in Table II, where the smallest class is class 13 with 916 data samples.

C. University of Pavia

A third HSI data used for experiments were the University of Pavia image shown in Fig. 4, which is an urban area surrounding the University of Pavia, Italy. It was recorded by the ROSIS-03 satellite sensor over an urban area surrounding the University of Pavia, Italy. It is of size $610 \times 340 \times 115$ with a spatial resolution of 1.3 m per pixel and a spectral coverage ranging from 0.43 to 0.86 μm with spectral resolution of 4 nm (12 most noisy channels were removed before experiments). Fig. 4(b) provides its ground-truth map of nine classes along with color class labels in Fig. 4(c). Table III summarizes the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

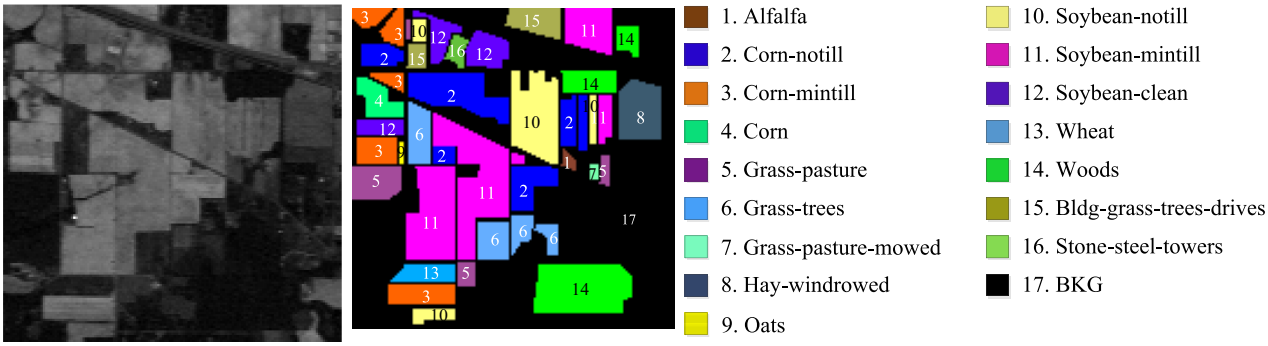


Fig. 2. Purdue's data and its GT with 16 classes. (a) Band 186 (2162.56 nm). (b) GT map. (c) Classes by colors.

TABLE I
CLASS LABELS OF PURDUE'S DATA

class 1 (46)	Alfalfa	class 7 (28)	grass/pasture-mowed	class 13 (205)	wheat
class 2 (1428)	corn-notill	class 8 (478)	hay-windrowed	class 14 (1265)	woods
class 3 (830)	corn-min	class 9 (20)	Oats	class 15 (386)	bldg-grass green-drives
class 4 (237)	corn	class 10 (972)	soybeans-notill	class 16 (93)	stone-steel towers
class 5 (483)	grass/pasture	class 11 (2455)	soybeans-min	class 17 (10249)	BKG
class 6 (730)	grass/trees	class 12 (593)	soybeans-clean		

TABLE II
CLASS LABELS OF CLASSES IN SALINAS

class 1 (2009)	Brocoli_green_weeds_1	class 7 (3579)	Celery	class 13 (916)	Lettuce_romaine_6wk
class 2 (3726)	Brocoli_green_weeds_2	class 8 (11271)	Grapes_untrained	class 14 (1070)	Lettuce_romaine_7w
class 3 (1976)	Fallow	class 9 (6203)	Soil_vinyard_develop	class 15 (7268)	Vinyard_untrained
class 4 (1394)	Fallow_rough_plow	class 10 (3278)	Corn_senesced_green_weeds	class 16 (1807)	Vinyard_vertical_trellis
class 5 (2678)	Fallow_smooth	class 11 (1068)	Lettuce_romaine_4wk	class 17 (56975)	BKG
class 6 (3959)	Stubble	class 12 (1927)	Lettuce_romaine_5wk		

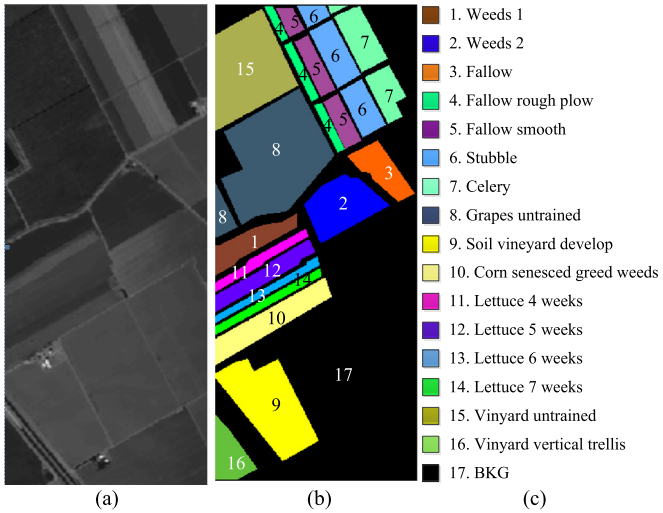


Fig. 3. Salinas and its GT with 16 classes. (a) Salinas scene. (b) Ground-truth image. (c) Classes by colors.

number of data samples in parentheses collected for each class. Like Salinas scene, this scene also has very large classes with only one small class less than 1000 data samples, class 9 with 947 samples.

D. Houston Data

A fourth and last image scene was Houston dataset, which collected over the campus area of University of Houston and its neighboring urban area as shown in Fig. 5. The image was captured by compact airborne spectrographic image (CASI) sensor in 2012. The dataset was published in 2013 IEEE GRSS Data Fusion Contest. It has a size of 349 × 1905 × 144 with 144 spectral bands ranging from 380 to 1050 nm and spatial resolution of 2.5 m. Table IV summarizes the number of data samples in parentheses collected for each class.

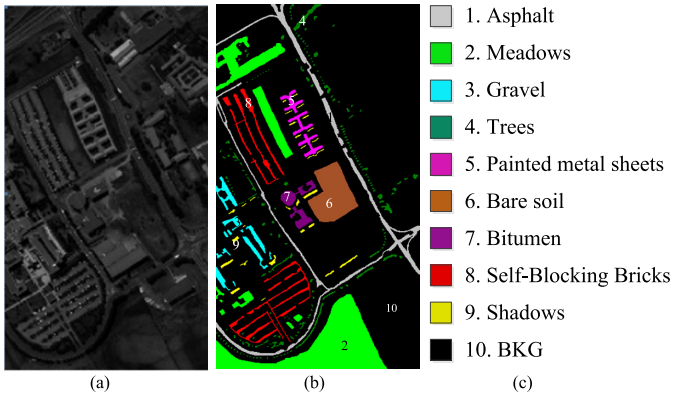


Fig. 4. University of Pavia and its GT with nine classes. (a) University of Pavia scene. (b) Ground-truth map. (c) Classes by colors.

X. EXPERIMENTAL RESULTS AND DISCUSSION

In this section, four widely used HIS datasets described in Section IX were used for experiments to evaluate the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

TABLE III
CLASS LABELS OF UNIVERSITY OF PAVIA

class 1 (6631)	Asphalt	class 5 (1345)	Painted metal sheets	class 9 (947)	Shadows
class 2 (18649)	Meadows	class 6 (5029)	Bare Soil	class 10 (164624)	BKG
class 3 (2099)	Gravel	class 7 (1330)	Bitumen		
class 4 (3064)	Trees	class 8 (3682)	Self-Blocking Bricks		

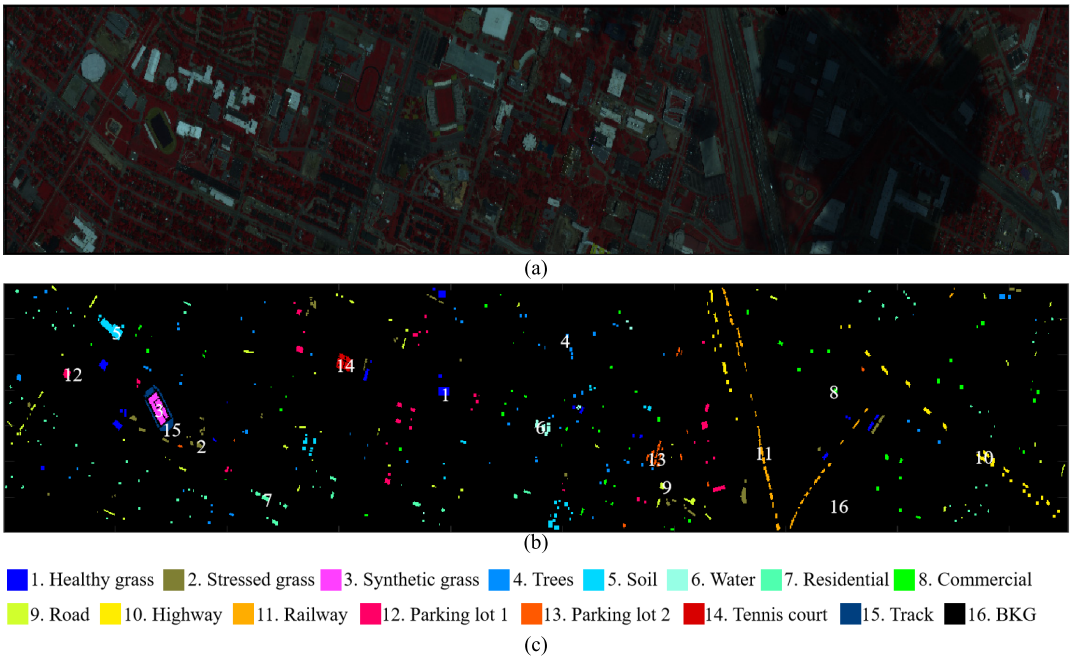


Fig. 5. Houston data and its GT with 15 classes. (a) False-color composite image. (b) GT map. (c) Classes by color.

TABLE IV
CLASS LABELS OF HOUSTON DATA WITH NUMBER OF DATA SAMPLES IN EACH CLASS

class 1 (1251)	Healthy grass	class 7 (1268)	Residential	class 13 (469)	Parking lot 2
class 2 (1254)	Stressed grass	class 8 (1244)	Commercial	class 14 (428)	Tennis court
class 3 (697)	Synthetic grass	class 9 (1252)	Road	class 15 (660)	Running track
class 4 (1244)	Trees	class 10 (1227)	Highway	class 16 (649816)	BKG
class 5 (1242)	Soil	class 11 (1235)	Railway		
class 6 (325)	Water	class 12 (1233)	Parking lot 1		

performance and effectiveness of IRTS-NKCEM/ NKLCMV. Several SS methods were selected and tested to conduct a comparative analysis as follows:

1) Iterative EPF (IEPF) developed in [18] and [19] because it improved EPF in [4] and outperformed many existing SS methods.
   Four versions of IEPF were developed. Only IEPF-G-g, which uses guided filters along with first principal component image as guidance image, was used for experiments because the other three were performed similarly and can be implemented similarly.
2) IRTS-EPF developed in [20].
   In general, IRTS-EPF can be considered as an advanced version of IEPF. The major difference between them is that IRTS-EPF randomly selects a new set of training samples to produce classification results in each iteration unlike IEPF, which only selects one set of training sample and used through all iterations. Because the concept of IRTS was originally introduced by IRTS-EPF, this method is also included for comparison.
3) IRTS-CEM proposed in [29].
   IRTS-CEM was originally to use IRTS to select training samples to calculate class sample mean as desired class signature instead of GT. However, as noted, CEM is a linear detector and cannot address nonlinear separability problem. In addition, IRTS-CEM requires a large amount of training samples (i.e., more than 25% of sampling rate for Indian Pines data) to achieve the classification accuracy of ICEM. Accordingly, IRTS-CEM is also included to see whether IRTS-NKCEM can resolve no-linear separability issue in IRTS-CEM.
4) The original version of ICEM developed in [26].
   However, we would like to point out that ICEM in [26] utilized entire GT to calculate class sample mean. It is indeed not fair to compare with other test methods.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

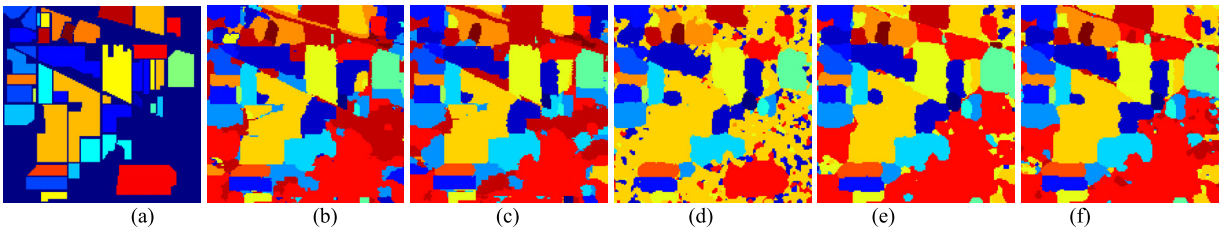


Fig. 6. Final binary classification maps for Indian Pines produced by five test methods. (a) GT, (b) IEPF-G-g ($P_{\mathrm{OA}} = 94.57\%$), (c) IRTS-EPF-G-g ($P_{\mathrm{OA}} = 98.02\%$), (d) IRTS-CEM ($P_{\mathrm{OA}} = 95.92\%$), (e) IRTS-NKCEM ($P_{\mathrm{OA}} = 99.56\%$), and (f) IRTS-NKLCMV ($P_{\mathrm{OA}} = 99.43\%$).

TABLE V
PARAMETERS USED FOR IRTS-NKCEM/NKLCMV FOR FOUR HSI DATASETS

	m_N/N	σ_{RBF}	β	ε	σ_{G}
Indian Pines	0.13	2	15	0.994	1.3
Salinas	0.013		15	0.994	2
U. Pavia	0.007		5	0.99	1
Houston	0.002		10	0.985	0.8

5) Two state-of-the-art SS feature extraction-based methods, OTVCA [50], [51] and IID [52].
These two methods extract SS features from original image cube and then followed up by SVM to perform classification.

Because IEPF and IRTS-EPF also produced real-valued class abundance fractional maps, their performances are evaluated by 3-D ROC-derived measures using various values of threshold τ. On the contrary, in order to calculate hard classification measures, P_{OA}, P_{AA}, and P_{PR}, MAP in (39) was used to convert the real-valued abundance fractional maps into binary classification maps. It should be noted that P_{PR} can be calculated by either excluding or including BKG. In this case, we use P_{PR} and $P_{\mathrm{PR}}(\mathrm{BKG})$ for distinction.

Table V summarizes the parameters used for IRTS-NKCEM and IRTS-NKLCMV where m_N/N is LRA sampling rate, σ_{RBF} is RBF parameter, β is the softmax parameter shown in (36), ε is threshold value of TI, and σ_{G} is standard deviation of Gaussian filters.

Another important issue is how to determine the size of training samples for each class. In this article, Kang's method [4] was used to allocate training samples for each class where the numbers of training samples and test samples for all image scenes are summarized in Table VI. It should be noted that in [20], the numbers of used training samples were 1025 for Indian Pines, 1083 for Salinas, and 900 for University of Pavia, which were found too many in our experiments because our proposed KCEM/KLCMV could do much better using fewer training samples. So, in this article, we reduced the size of training samples to 500 for three image scenes. As for Houston, the number of training samples was set to 750 as was done in [24], where the training sample size of each class was set to 50.

A. Purdue Indiana Indian Pines

The first image scene utilized for experiments is Purdue Indiana Indian Pines. Fig. 6 shows the classification maps produced by five test methods where the best one was IRTS-NKCEM ($P_{\mathrm{OA}} = 99.56\%$) and the worst result was IEPF-G-g with ($P_{\mathrm{OA}} = 94.57$). Table VII summarizes the hard classification measures, $P_{\mathrm{OA}}/P_{\mathrm{AA}}/P_{\mathrm{PR}}$, and the soft classification measures produced by 3-D ROC analysis are summarized in Table VIII.

From Table VII, we can see that the worst binary classification result was produced by IEPF-G-g because IEPF-G-g did not used IRTS to capture the information provided by different set of training samples. For the other four IRTS-based classifiers, IRTS-CEM was the worst one with $P_{\mathrm{OA}} = 95.66\%$, $P_{\mathrm{AA}} = 99.19\%$, and $P_{\mathrm{APR}} = 95.27\%$. On the contrary, the best two binary classification maps were created using IRTS-NKCEM and IRTS-NKLCMV with $P_{\mathrm{OA}}/P_{\mathrm{AA}}/P_{\mathrm{APR}} =$ 95.48%/99.71%/99.34% and 99.50%/99.71%/99.32%. Particularly, the results of IRTS-EPF-G-g in [20] already achieved very high accuracies. IRTS-NKCEM/NKLCMV was able to further improve nearly 1.2% of P_{OA} and P_{AA} from IRTS-EPF-g, which is very significant.

Furthermore, Table VIII summarizes the soft classification measures using 3-D ROC analysis. Overall, the highest $\mathrm{O/AAUC}_{(\mathrm{D,F})}$ was also produced using IRTS-NKCEM with $\mathrm{O/AAUC}_{(\mathrm{D,F})} = 0.9969/0.9985$ and IRTS-NKLCMV was the second best one with $\mathrm{O/AAUC}_{(\mathrm{D,F})} = 0.9966/0.9984$. On the contrary, IRTS-EPF-G-g and IEPF-G-g both generated better $\mathrm{O/AAUC}_{(\mathrm{D},\tau)}$ than IRTS-NKCEM/NKLCMV. However, a classifier with large $\mathrm{O/AAUC}_{(\mathrm{D},\tau)}$ value is not necessarily effective because it may also produce high $\mathrm{O/AAUC}_{(\mathrm{F},\tau)}$ as well. As the matter of fact, IRTS-EPF-G-g and IEPF-G-g produced nearly five times worse of $\mathrm{O/AAUC}_{(\mathrm{F},\tau)}$ than IRTS-NKCEM/NKLCMV because of the lack of BKG suppressibility. In this case, we would rather utilize $\mathrm{O/AAUC}_{\mathrm{SNPR}}$ to demonstrate the performance of classification results because $\mathrm{AUC}_{\mathrm{SNPR}}$ jointly considered both $\mathrm{AUC}_{(\mathrm{F},\tau)}$ and $\mathrm{AUC}_{(\mathrm{D},\tau)}$. As we can see, the $\mathrm{AAUC}_{\mathrm{SNPR}}$ produced by IRTS-EPF-G-g was nearly four times worse than by IRTS-NKCEM. The classification results also demonstrated that IRTS-CEM was the worst one among all test methods in terms of both hard and soft classification measures because CEM cannot resolve nonlinear separability problems.

Except for the classification measures, Table VII summarizes the computing times for all test methods. IRTS-CEM was the most efficient one because it conducted linear operation. On the contrary, two EPF-based methods used SVM as spectral classifier that requires kernelizing each training sample, which resulted in a computing time longer than CEM. NKCEM/NKLCMV were the slowest because they must kernelize all data samples. Interestingly,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

TABLE VI

NUMBER OF TRAINING SAMPLES AND TEST SAMPLES ALLOCATED INTO EACH CLASS FOR FOUR BENCHMARK DATASETS

Class	Indian Pines		Salinas		University of Pavia		Houston	
	n_{training}	n_{test}	n_{training}	n_{test}	n_{training}	n_{test}	n_{training}	n_{test}
1	23	23	31	1978	56	6575	50	1201
2	36	1391	31	3695	56	18593	50	1204
3	35	792	31	1945	55	2044	50	647
4	32	203	31	1363	56	3008	50	1194
5	32	448	31	2647	56	1289	50	1192
6	35	696	31	3928	55	4974	50	275
7	14	14	31	3548	56	1274	50	1218
8	32	444	33	11238	55	3627	50	1194
9	10	10	32	6171	55	892	50	1202
10	36	937	32	3246			50	1177
11	43	2416	31	1037			50	1185
12	34	560	31	6171			50	1183
13	33	172	31	3246			50	419
14	40	1227	31	1037			50	378
15	34	354	31	1896			50	610
16	31	62	31	885				
Total	500	9749	500	54031	500	42276	750	14279

TABLE VII

AVERAGED P_A, P_{PR}, P_{OA}, P_{AA}, AND P_{APR} RESULTING FROM FIVE TIMES IEPF-G-g, IRTS-EPF-G-g, IRTS-CEM, IRTS-NKCEM, AND IRTS-NKLCMV FOR PURDUE INDIANA PINES DATA

n_{training}	**IEPF-G-g** 500		**IRTS-EPF-G-g** 500		**IRTS-CEM** 500		**IRTS-NKCEM** 500		**IRTS-NKLCMV** 500	
Class	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$
1	98.70 (1.19)	94.84 (11.5)	98.70 (1.19)	**100** (0)	99.57 (0.97)	**100** (0)	**100** (0)	**100** (0)	**100** (0)	**100** (0)
2	91.57 (4.23)	93.96 (5.53)	97.02 (0.42)	98.11 (1.01)	93.90 (2.76)	93.88 (2.46)	**98.96** (0.44)	**99.48** (0.19)	98.4 (0.82)	99.37 (0.40)
3	95.25 (2.52)	94.15 (6.55)	98.60 (0.82)	99.59 (0.57)	93.45 (4.35)	97.71 (1.03)	**99.81** (0.20)	99.24 (0.83)	99.71 (0.25)	**99.43** (0.53)
4	99.92 (0.19)	78.93 (17.0)	**100** (0)	98.02 (1.19)	99.75 (0.38)	94.32 (2.53)	99.92 (0.19)	99.92 (0.19)	**100** (0)	99.58 (0.42)
5	94.66 (3.72)	98.72 (1.41)	98.88 (1.95)	99.01 (0.50)	95.36 (1.53)	99.22 (1.74)	98.72 (1.23)	99.84 (0.37)	**99.42** (0.51)	**99.10** (0.89)
6	99.32 (0.61)	99.08 (0.62)	99.70 (0.18)	99.76 (0.34)	99.15 (0.56)	97.04 (1.43)	99.94 (0.08)	99.43 (0.68)	**99.97** (0.06)	99.81 (0.36)
7	97.14 (1.60)	98.67 (2.98)	96.43 (0)	**100** (0)	**100** (0)	93.84 (7.54)	**100** (0)	97.93 (1.89)	**100** (0)	97.93 (1.89)
8	**100** (0)	**100** (0)	**100** (0)	**100** (0)	98.83 (0.88)	**100** (0)	**100** (0)	**100** (0)	**100** (0)	**100** (0)
9	**100** (0)	**100** (0)	**100** (0)	**100** (0)	**100** (0)	76.14 (18.1)	**100** (0)	**100** (0)	**100** (0)	99.05 (2.13)
10	89.05 (6.51)	92.82 (5.32)	97.57 (2.21)	93.63 (2.47)	91.24 (4.30)	93.34 (4.47)	**99.49** (0.70)	98.72 (0.64)	99.40 (0.51)	**99.15** (0.79)
11	93.10 (2.41)	96.20 (1.96)	96.70 (0.88)	98.55 (0.64)	96.00 (1.34)	92.96 (2.05)	99.12 (0.37)	**99.67** (0.31)	**99.49** (0.30)	99.47 (0.35)
12	95.08 (4.08)	87.97 (7.52)	99.53 (0.14)	96.83 (2.27)	92.92 (3.68)	98.75 (0.99)	**99.39** (0.28)	99.53 (0.51)	99.33 (0.36)	**99.86** (0.14)
13	99.31 (0.27)	**100** (0)	99.12 (0.22)	**100** (0)	99.61 (0.87)	96.99 (2.00)	**100** (0)	99.71 (0.64)	99.9 (0.22)	**100** (0)
14	97.74 (4.09)	99.49 (0.24)	99.49 (0.57)	99.65 (0.12)	97.04 (1.24)	98.83 (0.47)	**100** (0)	99.56 (0.28)	**100** (0)	99.68 (0.16)
15	100 (0)	90.47 (9.02)	**100** (0)	96.19 (2.00)	98.81 (1.34)	98.26 (1.35)	**100** (0)	99.79 (0.33)	**100** (0)	99.95 (0.12)
16	98.49 (0.59)	97.06 (1.67)	95.91 (1.77)	98.24 (0.59)	99.35 (0.96)	93.07 (5.44)	**100** (0)	96.71 (1.92)	99.78 (0.48)	96.72 (2.35)
P_{OA}	94.84 (1.75)		98.22 (0.25)		95.66 (0.34)		99.48 (0.16)		**99.50** (0.10)	
P_{AA}	96.83 (0.85)		98.60 (0.26)		97.19 (0.22)		**99.71** (0.12)		**99.71** (0.04)	
P_{APR}	95.15 (1.93)		98.60 (0.19)		95.27 (1.20)		**99.34** (0.23)		99.32 (0.25)	
P_{PR} (BKG)	46.23 (0.85)		47.88 (0.12)		46.63 (0.17)		48.49 (0.08)		**48.50** (0.04)	
Iteration	7 (2.32)		10.40 (2.30)		16 (0)		15.80 (0.44)		13.4. (2.30)	

although the classification performance of IRTS-NKCEM and IRTS-NKLCMV were closed, IRTS-NKLCMV were still faster than IRTS-NKCEM. This was because NKLCMV classified all classes at once unlike NKCEM, which only classified one class a time.

To better demonstrate the soft-classification results produced by five test methods, Fig. 7 shows the real-valued class abundance fractional maps produced by all classifiers along with the complete GT of each class. Because of the limited of space, we only showed classes 7, 10, 11, and 16 where class 10 (972 samples) and class 11 (2455 samples) can interpret the real-value classification results of large class. On the contrary, class 7 (28 samples) and class 16 (93 samples) could be used to demonstrate the classification results of small classes.

By visual inspection, even though IEPF-G-g and IRTS-EPF-G-g also show to be similar with GT map, their misclassification error in terms of BKG were clearly much higher than IRTS-NKCEM/IRTS-NKLCMV. This was because IEPF-G-g and IRTS-EPF-G-g used SVM as spectral classifier, which did not deal with BKG. On the contrary, we could also observe that IRTS-CEM was only able to perform well on small classes such as class 7 and class 16. For the large class such as class 11, IRTS-CEM not only produced the worst classification accuracy but also had weak BKG suppressibility. This was because the large classes are very easy to be interfered by BKG. In this case, kernel would be very helpful to be used to enhance the separability between class data samples and BKG samples thus improved BKG suppressibility. This can demonstrate why IRTS-NKCEM and IRTS-NKLCMV both showed better classification performance than IRTS-CEM.

B. Salinas

The second image scene is Salinas. Tables IX and X summarize the results calculated by hard and soft classification measures of Salinas data. Fig. 8 shows the hard classification

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII

AVERAGED $AUC_{(D,F)}$, $AUC_{(F,\tau)}$, $O/AAUC_{(D,F)}$ $O/AAUC_{(D,\tau)}$, $O/AAUC_{(F,\tau)}$, AND O/AAUCSNPR RESULTING FROM FIVE TIMES IEPF-G-g, IRTS-EPF-G-g, IRTS-CEM, IRTS-NKCEM, AND IRTS-NKLCMV FOR SALINAS DATA

n_{training}	**IEPF-G-g** 500		**IRTS-EPF-G-g** 500		**IRTS-CEM** 500		**IRTS-NKCEM** 500		**IRTS-NKLCMV** 500	
Class	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$
1	0.9994	0.0036	0.9996	0.0025	**0.9999**	0.0069	**0.9999**	0.0014	**0.9999**	**0.0013**
2	0.9797	0.0413	0.9896	0.0374	0.9821	0.0599	**0.9959**	**0.0122**	0.9957	0.0155
3	0.9948	0.0201	0.9969	0.0185	0.9900	0.0351	0.9981	0.0083	**0.9984**	**0.0080**
4	0.9990	0.0157	0.9996	0.0045	0.9996	0.0167	**0.9999**	**0.0024**	0.9998	0.0026
5	0.9691	0.0474	0.9869	0.0576	0.9947	0.0287	**0.9985**	**0.0076**	**0.9985**	0.0091
6	0.9955	0.0274	0.9951	0.0281	0.9977	0.0372	**0.9981**	**0.0087**	**0.9980**	0.0114
7	0.9999	0.0025	0.9999	0.0016	0.9999	0.0082	**1.0000**	**0.0011**	**1.0000**	**0.0011**
8	0.9987	0.0083	0.9989	0.0074	0.9996	0.0244	**0.9996**	0.0032	**0.9996**	0.0042
9	0.9997	0.0026	0.9997	0.0025	0.9997	0.0112	**0.9999**	**0.0018**	**0.9999**	0.0021
10	0.9602	0.0193	0.9965	0.0216	0.9773	0.0435	0.9974	**0.0101**	**0.9979**	0.0131
11	0.9884	0.0321	0.9916	0.0365	0.9616	0.0817	**0.9939**	**0.0132**	0.9931	0.0205
12	0.9944	0.0256	0.9964	0.0332	0.9916	0.0269	**0.9990**	**0.0052**	0.9989	0.0066
13	**0.9999**	0.0030	**0.9999**	0.0026	0.9997	0.0165	0.9997	0.0032	0.9997	**0.0025**
14	0.9657	0.1306	0.9721	0.1253	0.9890	0.0481	**0.9970**	**0.0186**	0.9962	0.0253
15	0.9629	0.1533	0.9651	0.1428	0.9989	0.0257	0.9996	**0.0048**	**0.9997**	0.0071
16	0.9984	0.0108	0.9987	0.0066	0.9998	0.009	**0.9999**	0.0016	**0.9999**	**0.0015**
$O/AAUC_{(D,F)}$	0.9824/0.9878		0.9899/0.9929		0.9830/0.9926		**0.9969/0.9985**		0.9966/0.9984	
$O/AAUC_{(D,\tau)}$	0.7793/0.7863		**0.7924/0.7989**		0.4420/0.5521		0.3870/0.5310		0.4224/0.5432	
$O/AAUC_{(F,\tau)}$	0.0454/0.0340		0.0455/0.0330		0.0494/0.0300		**0.0105/0.0065**		0.0143/0.0082	
$O/AAUC_{SNPR}$	17.1875/23.1558		17.4606/24.1962		8.9626/18.4028		**37.20/82.98**		30.97/69.57	

TABLE IX

AVERAGED P_A, P_{PR}, P_{OA}, P_{AA}, AND P_{APR} RESULTING FROM FIVE TIMES IEPF-G-g, IRTS-EPF-G-g, IRTS-CEM, IRTS-NKCEM, AND IRTS-NKLCMV FOR SALINAS DATA

n_{training}	**IEPF-G-g** 500		**IRTS-EPF-G-g** 500		**IRTS-CEM** 500		**IRTS-NKCEM** 500		**IRTS-NKLCMV** 500	
class	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$
1	**100** (0)	**100** (0)	99.97 (0.07)	**100** (0)	99.47 (0.70)	99.97 (0.07)	**100** (0)	**100** (0)	**100** (0)	**100** (0)
2	99.97 (0.05)	**100** (0)	99.83 (0.19)	**100** (0)	97.15 (3.46)	99.70 (0.30)	**100** (0)	**100** (0)	**100** (0)	**100** (0)
3	99.64 (0.36)	99.98 (0.04)	99.31 (0.59)	**100** (0)	99.56 (0.51)	99.95 (0.12)	**100** (0)	**100** (0)	**100** (0)	**100** (0)
4	99.46 (0.44)	97.76 (0.34)	99.73 (0.18)	97.82 (0.39)	98.92 (0.95)	94.44 (2.58)	99.80 (0.16)	**99.39** (0.41)	**99.94** (0.08)	98.94 (0.34)
5	98.47 (0.19)	**100** (0)	98.61 (0.13)	**100** (0)	95.15 (2.22)	99.91 (0.11)	**99.75** (0.19)	99.90 (0.08)	99.49 (0.17)	99.97 (0.04)
6	99.91 (0.07)	**100** (0)	99.83 (0.21)	**100** (0)	99.78 (0.43)	99.49 (0.27)	**99.99** (0.01)	**100** (0)	99.98 (0.04)	99.99 (0.01)
7	99.58 (0.07)	99.98 (0.03)	99.55 (0.08)	99.98 (0.03)	98.99 (0.92)	99.52 (0.59)	**100** (0)	**99.99** (0.01)	**100** (0)	99.98 (0.03)
8	95.9 (2.69)	96.11 (3.47)	96.09 (2.65)	**99.42** (0.47)	94.40 (2.91)	93.70 (2.57)	97.89 (0.44)	98.17 (0.83)	**98.37** (1.11)	98.71 (0.84)
9	99.4 (0.45)	99.89 (0.11)	98.85 (0.35)	99.96 (0.06)	98.70 (0.60)	99.50 (0.59)	**100** (0)	99.96 (0.06)	**100** (0)	**99.98** (0.02)
10	98.52 (2.61)	91.53 (4.61)	99.92 (0.11)	88.84 (1.84)	98.12 (1.83)	99.59 (0.36)	99.91 (0.14)	99.79 (0.28)	**99.92** (0.08)	**99.75** (0.24)
11	98.67 (0.63)	**100** (0)	98.95 (0.39)	**100** (0)	99.87 (0.21)	99.76 (0.44)	**100** (0)	**100** (0)	**100** (0)	**100** (0)
12	**100** (0)	**100** (0)	99.98 (0.04)	**100** (0)	98.98 (0.58)	99.97 (0.07)	**100** (0)	**100** (0)	99.99 (0.02)	**100** (0)
13	98.99 (0.48)	99.22 (1.74)	99.13 (0.25)	**100** (0)	98.38 (1.56)	99.6 (0.64)	**99.98** (0.05)	**100** (0)	99.96 (0.1)	99.98 (0.05)
14	96.88 (3.07)	98.91 (0.74)	97.96 (1.01)	99.24 (0.2)	99.21 (0.75)	98.97 (1.25)	**99.87** (0.21)	**99.96** (0.05)	99.85 (0.34)	99.81 (0.27)
15	94.39 (3.86)	95.85 (3.22)	98.72 (0.71)	96.72 (2.91)	94.09 (3.26)	90.56 (4.07)	97.13 (1.32)	96.81 (0.71)	**97.98** (1.31)	**97.60** (1.68)
16	98.02 (0.44)	**100** (0)	98.2 (0.56)	**100** (0)	95.02 (5.47)	99.93 (0.15)	99.99 (0.03)	**100** (0)	99.98 (0.03)	**100** (0)
P_{OA}	97.92 (0.75)		98.58 (0.54)		96.94 (0.53)		99.15 (0.17)		**99.35** (0.30)	
P_{AA}	98.61 (0.34)		99.04 (0.16)		97.86 (0.36)		99.65 (0.07)		**99.72** (0.13)	
P_{APR}	98.70 (0.36)		98.87 (0.30)		98.41 (0.17)		99.62 (0.05)		**99.67** (0.13)	
P_{PR}(BKG)	47.71 (0.36)		48.03 (0.26)		47.23 (0.26)		48.31 (0.08)		**48.40** (0.14)	
Iteration	5.00 (1.22)		7.00 (2.00)		15 (0)		14.4 (2.60)		13.6 (0.54)	

maps produced by five test methods where the best one was IRTS-NKCEM with $P_{OA} = 99.08\%$.

Similar to the Indian Pines results, the worst P_{OA} was also produced by IRTS-CEM with nearly 2% and 1% lower than IRTS-EPF-G-g and IEPF-G-g. On the contrary, the best binary classification map was generated using IRTS-NKLCMV with $P_{OA}/P_{AA} = 99.35/99.72$, which was slightly better than IRTS-NKCEM with $P_{OA}/P_{AA} = 99.15/99.65$.

In addition, the results in Table X also reveal that IRTS-NKCEM/NKLCMV was the most effective one with the highest $O/AAUC_{(D,F)}$ and $O/AAUC_{SNPR}$. Particularly, $O/AAUC_{SNPR}$ produced by IRTS-NKCEM/NKLCMV were nearly 3 and 6 times better than IRTS-EPF-G-g and IRTS-CEM. It is also noted that the IRTS-CEM generated the worst $O/AAUC_{SNPR}$, which indicated that IRTS-CEM was not effective as its nonlinear version, IRTS-NKCEM.

C. University of Pavia

Tables XI and XII summarize the classification performance of University of Pavia image scene where the best results are bold face and the classification maps of five test methods are as shown in Fig. 9.

As summarized in Tables XI and XII, if we only use the results calculated by hard classification measures in Table XI,

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

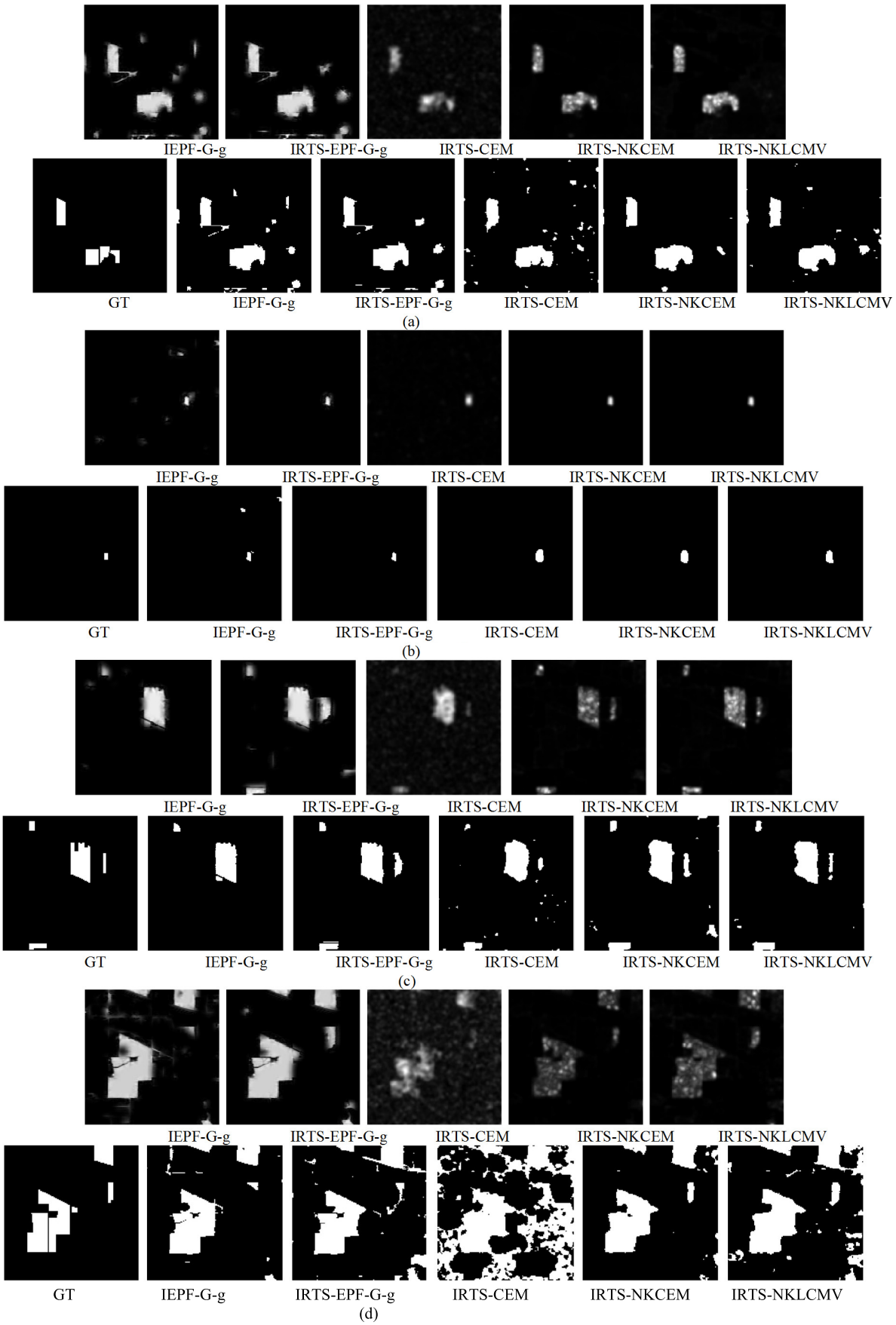


Fig. 7. Real-valued class abundance fractional maps and binary classification maps of classes 6, 7, 10, 11, and 16 produced by virous SS-classifier (a) class 6, (b) class 7, (c) class 10, (d) class 11, and (e) class 16, respectively.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

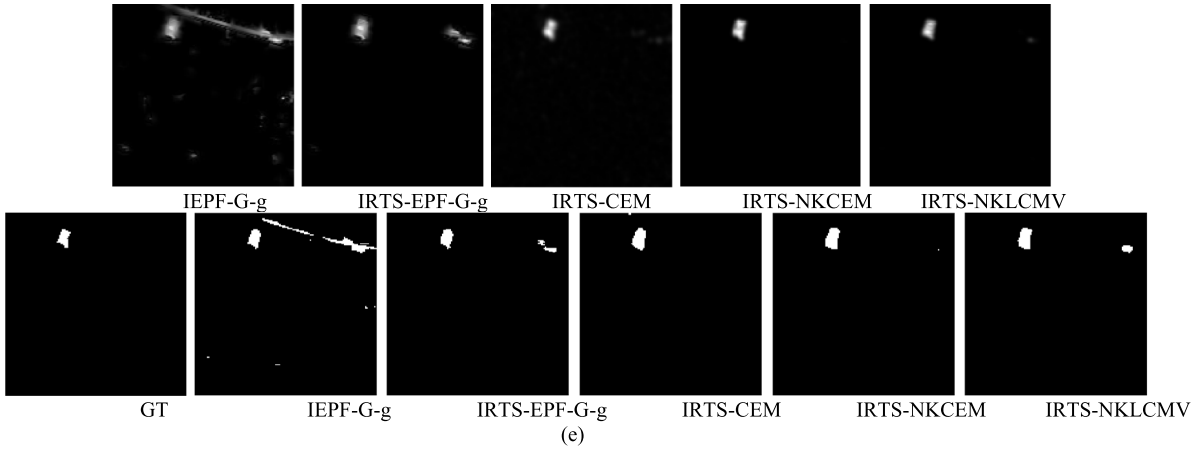


Fig. 7. *(Continued.)* Real-valued class abundance fractional maps and binary classification maps of classes 6, 7, 10, 11, and 16 produced by virous SS-classifier (a) class 6, (b) class 7, (c) class 10, (d) class 11, and (e) class 16, respectively.

TABLE X

AVERAGED $AUC_{(D,F)}$, $AUC_{(F,\tau)}$, $O/AAUC_{(D,F)}$ $O/AAUC_{(D,\tau)}$, $O/AAUC_{(F,\tau)}$, AND $O/AAUCSNPR$ RESULTING FROM FIVE TIMES IEPF-G-g, IRTS-EPF-G-g, IRTS-CEM, IRTS-NKCEM, AND IRTS-NKLCMV FOR SALINAS DATA

n_{training}	**IEPF-G-g** 500		**IRTS-EPF-G-g** 500		**IRTS-CEM** 500		**IRTS-NKCEM** 500		**IRTS-NKLCMV** 500	
class	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$
1	**0.9999**	0.0056	**0.9999**	0.0056	0.9997	0.0182	0.9994	**0.0030**	0.9995	0.0031
2	0.9983	0.0118	0.9983	0.0118	0.9981	0.0234	0.9992	**0.0031**	**0.9994**	0.0034
3	0.9394	0.1275	0.9394	0.1275	0.9985	0.0191	**0.9993**	**0.0047**	**0.9993**	0.0048
4	0.9864	0.0263	0.9864	0.0263	0.9981	0.0190	**0.9996**	0.0056	0.9993	**0.0051**
5	0.9921	0.0240	0.9921	0.0240	0.9975	0.0216	0.9982	**0.0077**	**0.9984**	0.0085
6	**0.9991**	0.0078	**0.9991**	0.0078	0.9988	0.0331	0.9987	0.0041	0.9986	**0.0040**
7	**0.9993**	0.0067	**0.9993**	0.0067	0.9979	0.0283	0.9986	**0.0042**	0.9986	**0.0042**
8	0.9919	0.0401	0.9919	0.0401	0.9701	0.0535	0.9946	0.0112	**0.9960**	**0.0093**
9	0.9652	0.0810	0.9652	0.0810	0.9966	0.0414	0.9979	**0.0093**	**0.9986**	0.0105
10	0.9615	0.1146	0.9615	0.1146	0.9992	0.0277	**0.9993**	**0.0040**	**0.9993**	0.0049
11	0.9844	0.0350	0.9844	0.0350	0.9997	0.0153	**0.9998**	**0.0027**	**0.9999**	**0.0027**
12	0.9899	0.0255	0.9899	0.0255	0.9995	0.0173	0.9995	0.0032	**0.9996**	0.0032
13	**0.9997**	0.007	**0.9997**	0.0070	0.9992	0.0145	0.9996	**0.0030**	**0.9997**	0.0032
14	0.9989	0.0073	0.9989	0.0073	0.9992	0.0127	**0.9997**	**0.0026**	**0.9997**	0.0030
15	**0.9986**	0.0085	**0.9986**	0.0085	0.9907	0.0514	0.9976	0.0089	0.9985	**0.0076**
16	**0.9998**	0.0029	**0.9998**	0.0029	0.9993	0.0153	0.9995	**0.0013**	0.9995	0.0020
$O/AAUC_{(D,F)}$	0.9879/0.9878		0.9881/0.9882		0.9914/0.9964		0.9978/0.9988		**0.9983**/**0.9990**	
$O/AAUC_{(D,\tau)}$	0.8751/0.8764		**0.8797**/**0.8793**		0.4640/0.5102		0.4502/0.5277		0.4473/0.5325	
$O/AAUC_{(F,\tau)}$	0.0362/0.0332		0.0372/0.0331		0.0352/0.0257		0.0068/**0.0049**		**0.0064**/0.0050	
$O/AAUC_{SNPR}$	25.46/26.38		24.88/26.54		13.19/19.83		69.14/**110.22**		**69.71**/107.50	

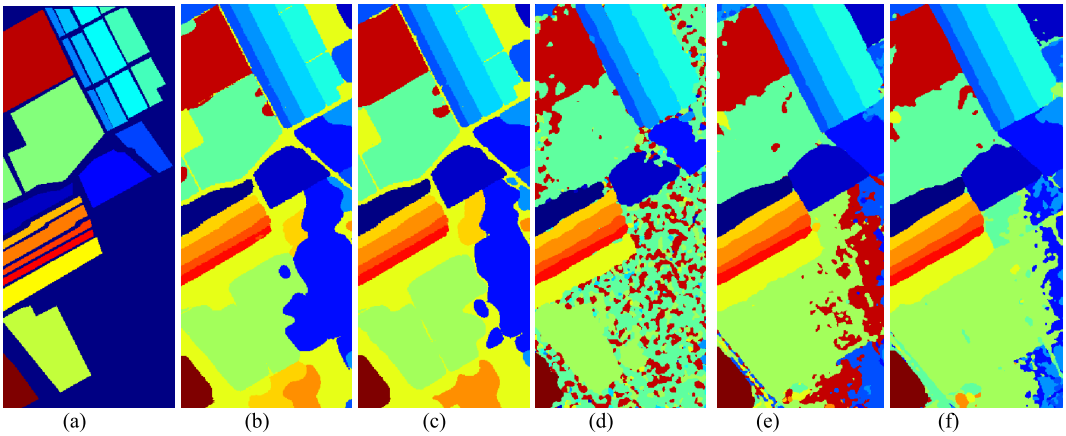
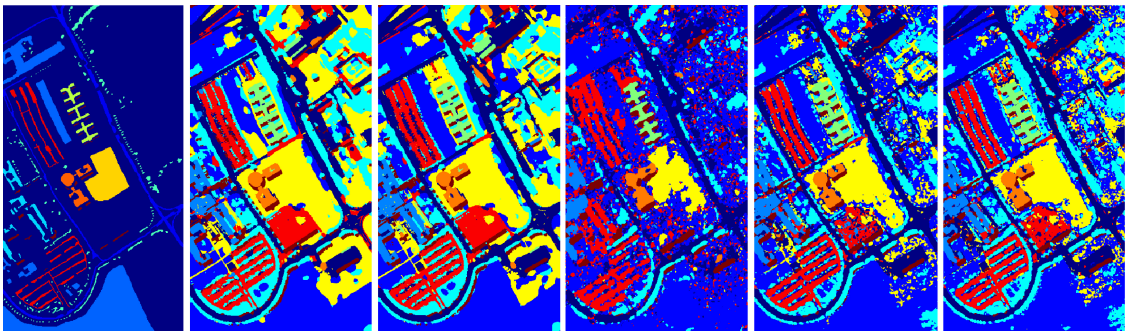


Fig. 8. Final binary classification maps for Salinas produced by five test methods. (a) GT, (b) IEPF-G-g (P_{OA} = 98.15%), (c) IRTS-EPF-G-g (P_{OA} = 98.51%), (d) IRTS-CEM (P_{OA} = 96.16%), (e) IRTS-NKCEM (P_{OA} = 99.08%), and (f) IRTS-NKLCMV (P_{OA} = 99.01%).

IRTS-EPF-G-g will perform better than IRTS-NKCEM. This showed that the hard classification measures were good for pure pixel-based classifiers. However, if we compare the results calculated by soft classification measures in Table XII, IRTS-NKCEM actually outperformed IRTS-EPF-G-g where IRTS-NKCEM not only produced higher $O/AAUC_{(D,F)}$ but

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.



(a) (b) (c) (d) (e) (f)

Fig. 9. Final binary classification maps for University of Pavia produced by five test methods. (a) GT, (b) IEPF-G-g ($P_{OA} = 97.44\%$), (c) IRTS-EPF-G-g ($P_{OA} = 99.32\%$), (d) IRTS-CEM ($P_{OA} = 90.32\%$), (e) IRTS-NKCEM ($P_{OA} = 99.24\%$), and (f) IRTS-NKLCMV ($P_{OA} = 99.01\%$).

TABLE XI

AVERAGED P_A, P_{PR}, P_{OA}, P_{AA}, AND P_{APR} RESULTING FROM FIVE TIMES IEPF-G-g, IRTS-EPF-G-g, IRTS-CEM, IRTS-NKCEM, AND IRTS-NKLCMV FOR UNIVERSITY OF PAVIA DATA

$n_{training}$	**IEPF-G-g** 500		**IRTS-EPF-G-g** 500		**IRTS-CEM** 500		**IRTS-NKCEM** 500		**IRTS-NKLCMV** 500	
class	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$
1	97.46 (1.69)	98.82 (0.24)	**99.27** (0.48)	98.63 (0.41)	87.61 (2.53)	92.15 (3.53)	98.86 (0.98)	99.51 (0.27)	98.73 (0.58)	**99.50** (0.11)
2	95.22 (3.04)	99.56 (0.22)	99.42 (0.41)	99.68 (0.09)	93.89 (0.65)	91.63 (0.65)	99.37 (0.11)	99.67 (0.17)	**99.52** (0.33)	**99.77** (0.09)
3	96.64 (2.60)	95.34 (2.56)	**99.06** (1.02)	**99.02** (1.23)	68.26 (2.12)	95.15 (3.56)	94.75 (1.87)	98.88 (0.72)	95.93 (2.25)	98.43 (0.93)
4	95.14 (1.17)	90.13 (5.11)	94.78 (1.32)	97.74 (2.55)	86.64 (6.12)	91.07 (1.85)	98.53 (0.52)	**98.49** (1.25)	**98.90** (0.41)	98.22 (1.90)
5	99.97 (0.04)	99.17 (0.24)	**100** (0)	99.22 (0.39)	99.76 (0.15)	99.85 (0.18)	**100** (0)	**99.99** (0.03)	**100** (0)	**99.99** (0.03)
6	99.55 (0.89)	89.72 (6.65)	**99.74** (0.52)	98.64 (0.87)	88.22 (3.63)	97.72 (1.84)	99.68 (0.33)	98.52 (0.65)	99.73 (0.31)	**99.31** (0.59)
7	**100** (0)	98.99 (2.27)	99.94 (0.13)	**100** (0)	97.56 (1.55)	99.92 (0.08)	99.79 (0.17)	**100** (0)	99.79 (0.27)	**100** (0)
8	97.74 (1.83)	95.54 (2.90)	**98.94** (0.73)	**98.42** (0.62)	87.16 (5.50)	67.00 (2.36)	98.63 (0.43)	95.31 (2.34)	98.42 (0.54)	95.57 (0.90)
9	**100** (0)	99.12 (0.80)	**100** (0)	99.12 (0.60)	96.85 (0.57)	97.60 (1.74)	99.89 (0.18)	**100** (0)	99.91 (0.14)	99.96 (0.09)
P_{OA}	96.76 (1.21)		99.09 (0.13)		90.26 (0.81)		99.02 (0.12)		**99.14** (0.16)	
P_{AA}	97.97 (0.28)		**99.02** (0.23)		89.55 (0.91)		98.83 (0.18)		98.99 (0.16)	
P_{APR}	96.27 (1.07)		98.94 (0.19)		92.46 (0.71)		98.93 (0.22)		**98.97** (0.24)	
P_{PR}(BKG)	19.96 (0.25)		20.44 (0.03)		18.61 (0.17)		20.44 (0.03)		**20.45 (0.03)**	
Iteration	4.20 (0.44)		4.8 (0.44)		16 (0)		15.20 (1.09)		15.60 (0.89)	

TABLE XII

AVERAGED $AUC_{(D,F)}$, $AUC_{(F,\tau)}$, $O/AAUC_{(D,F)}$ $O/AAUC_{(D,\tau)}$, $O/AAUC_{(F,\tau)}$, AND $O/AAUC_{SNPR}$ RESULTING FROM FIVE TIMES IEPF-G-g, IRTS-EPF-G-g, IRTS-CEM, IRTS-NKCEM, AND IRTS-NKLCMV FOR UNIVERSITY OF PAVIA DATA

$n_{training}$	**IEPF-G-g** 500		**IRTS-EPF-G-g** 500		**IRTS-CEM** 500		**IRTS-NKCEM** 500		**IRTS-NKLCMV** 500	
class	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$
1	0.9520	0.1197	0.9582	0.1251	0.9227	0.0427	0.9829	0.0190	**0.9832**	**0.0183**
2	0.9585	0.1417	0.9750	0.1409	0.8953	0.0548	**0.9790**	0.0171	0.9774	**0.0147**
3	0.9921	0.0186	0.9951	0.0202	0.9477	0.0194	**0.9978**	**0.0062**	**0.9978**	0.0072
4	0.9307	0.1462	0.9366	0.1358	0.9466	0.0312	0.9794	**0.0154**	**0.9802**	0.0158
5	0.9973	0.0100	0.9965	0.0117	0.9995	0.0111	**0.9996**	**0.0033**	**0.9996**	0.0043
6	0.9718	0.1290	0.9761	0.1277	0.9740	0.0172	**0.9952**	**0.0105**	0.9951	0.0116
7	0.9983	0.0095	0.9982	0.0098	0.9983	0.0074	**0.9994**	**0.0039**	0.9993	0.0044
8	0.9757	0.0622	0.9803	0.0572	0.9362	0.0387	**0.9924**	0.0124	0.9923	**0.0110**
9	0.9964	0.0541	0.9964	0.0573	0.9928	0.0188	0.9977	**0.0057**	**0.9978**	0.0060
$O/AAUC_{(D,F)}$	0.9635/0.9747		0.9731/0.9792		0.9272/0.9570		**0.9853/0.9915**		0.9847/0.9914	
$O/AAUC_{(D,\tau)}$	0.7454/0.7429		**0.7600/0.7534**		0.3002/0.3789		0.2488/0.3632		0.2401/0.3671	
$O/AAUC_{(F,\tau)}$	0.1140/0.0768		0.1134/0.0762		0.0401/0.0268		**0.0145/0.0104**		0.0134/0.0104	
$O/AAUC_{SNPR}$	6.5772/9.6774		6.7134/9.8911		7.53/14.17		17.32/35.25		**17.88/35.68**	

also generated three times better of $O/AAUC_{SNPR}$ than IRTS-EPF-G-g. This is because the traditional hard classification measures did not consider misclassification errors resulting from BKG samples.

Another interesting fact is that IRTS-CEM produced the worst classification performance in terms of both hard and soft classification measures. This may be because the University of Pavia data did not provide sufficient spectral information for linear classifiers. On the contrary, the results clearly showed that using kernels to project data samples into high-dimensional feature space indeed improved classification performance significantly where IRTS-NKCEM/NKLCMV was able to improve nearly 9% of P_{OA}/P_{AA} from IRTS-CEM.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

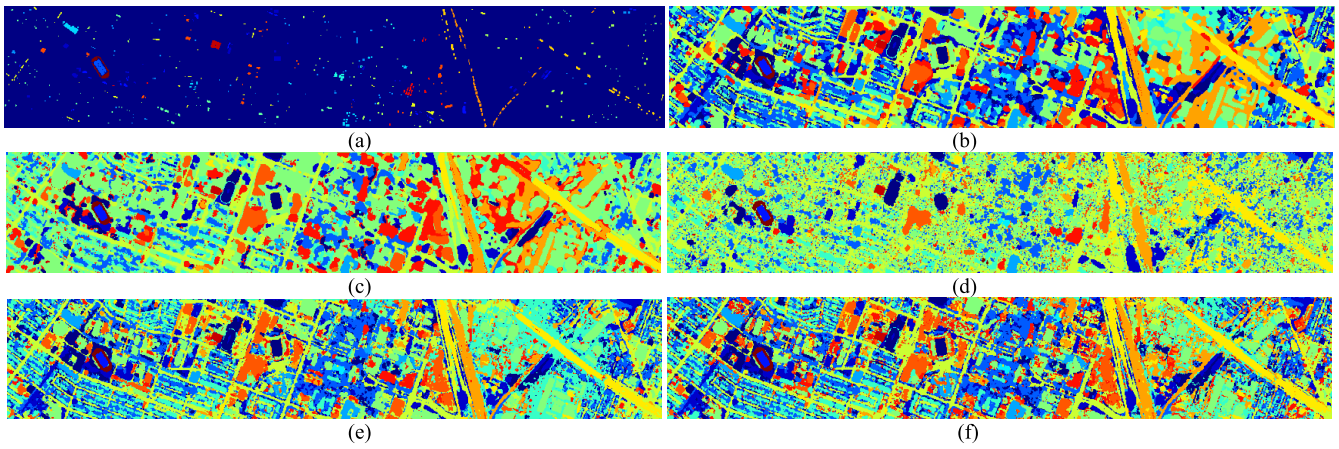


Fig. 10. Final binary classification maps for Houston data produced by five test methods. (a) GT, (b) IEPF-G-g (P_{OA} = 97.21%), (c) IRTS-EPF-G-g (P_{OA} = 97.87%), (d) IRTS-CEM (P_{OA} = 87.60%), (e) IRTS-NKCEM (P_{OA} = 98.90%), and (f) IRTS-NKLCMV (P_{OA} = 98.63%).

TABLE XIII

AVERAGED P_A, P_{PR}, P_{OA}, P_{AA}, AND P_{APR} RESULTING FROM FIVE TIMES IEPF-G-g, IRTS-EPF-G-g, IRTS-CEM, IRTS-NKCEM, AND IRTS-NKLCMV FOR HOUSTON DATA

$n_{training}$	**IEPF-G-g** 500		**IRTS-EPF-G-g** 500		**IRTS-CEM** 500		**IRTS-NKCEM** 500		**IRTS-NKLCMV** 500	
class	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$	$P_A(C_i)$	$P_{PR}(C_i)$
1	98.21 (1.44)	97.53 (1.85)	97.77 (1.12)	98.75 (0.55)	84.45 (1.76)	93.53 (1.85)	**99.45** (1.01)	98.85 (0.60)	99.18 (0.88)	**99.24** (0.41)
2	97.99 (1.69)	98.60 (0.97)	98.67 (0.51)	99.38 (0.54)	90.11 (2.63)	97.36 (1.41)	99.31 (0.44)	**99.88** (0.21)	**99.49** (0.26)	**99.81** (0.18)
3	99.87 (0)	99.67 (0.11)	99.92 (0.07)	99.52 (0.13)	99.87 (0.18)	99.95 (0.11)	99.37 (0.74)	**100** (0)	**99.92** (0.07)	**99.97** (0.06)
4	98.43 (1.41)	97.66 (0.73)	99.27 (0.81)	**99.34** (0.73)	85.19 (6.81)	91.32 (5.77)	**99.92** (0.10)	**99.46** (0.85)	99.90 (0.07)	99.24 (0.89)
5	**100** (0)	98.59 (1.36)	99.46 (0.85)	99.94 (0.14)	94.33 (5.64)	99.98 (0.04)	99.95 (0.10)	99.92 (0.17)	99.44 (1.15)	**100** (0)
6	**98.35** (0.45)	96.58 (6.03)	98.06 (0.34)	**100** (0)	89.68 (4.51)	99.48 (0.60)	93.51 (3.15)	**100** (0)	92.10 (6.14)	**100** (0)
7	96.10 (2.16)	97.22 (2.07)	98.06 (2.08)	**99.89** (0.09)	82.70 (2.55)	83.90 (4.34)	98.78 (1.35)	97.22 (1.23)	**99.10** (0.67)	97.50 (1.49)
8	86.13 (3.72)	98.50 (1.17)	97.70 (1.38)	92.56 (8.12)	76.29 (11.7)	72.02 (17.7)	**98.45** (0.92)	98.27 (1.25)	97.46 (1.33)	**99.31** (0.83)
9	94.87 (2.11)	95.93 (2.81)	96.40 (3.04)	**98.45** (1.68)	91.66 (5.84)	61.17 (14.1)	**97.79** (1.28)	98.09 (1.01)	97.73 (0.97)	96.19 (1.66)
10	98.84 (1.66)	97.81 (1.31)	99.38 (1.04)	**99.31** (0.65)	90.77 (6.18)	89.25 (8.91)	99.11 (0.90)	98.85 (1.27)	**99.65** (0.33)	98.12 (1.11)
11	98.61 (1.31)	96.69 (2.68)	**98.81** (1.12)	**99.55** (0.71)	75.42 (3.15)	97.85 (1.46)	98.21 (1.10)	98.72 (1.04)	97.61 (0.97)	97.61 (1.47)
12	94.60 (5.46)	97.43 (1.89)	97.98 (0.85)	**99.77** (0.24)	72.61 (4.98)	90.52 (11.1)	**98.82** (0.82)	98.03 (0.81)	97.27 (1.52)	98.06 (0.60)
13	95.13 (1.86)	78.49 (9.12)	**98.92** (0.91)	92.20 (6.13)	83.80 (5.80)	96.58 (1.56)	96.39 (1.26)	**96.98** (2.55)	96.04 (2.57)	96.89 (1.95)
14	99.89 (0.10)	96.96 (1.7)	99.65 (0.25)	**100** (0)	99.77 (0.25)	**100** (0)	98.87 (1.42)	**100** (0)	**100** (0)	**100** (0)
15	99.60 (0.10)	99.87 (0)	99.40 (0.14)	**99.92** (0.07)	99.37 (0.75)	99.43 (0.54)	**99.65** (0.48)	99.62 (0.41)	**99.97** (0.06)	**99.92** (0.07)
P_{OA}	96.78 (0.60)		98.50 (0.65)		86.24 (1.11)		**98.79** (0.14)		98.59 (0.26)	
P_{AA}	97.11 (0.44)		**98.63** (0.52)		87.73 (1.19)		98.50 (0.20)		98.33 (0.46)	
P_{APR}	96.50 (0.85)		98.57 (0.58)		91.49 (0.91)		**98.92** (0.17)		98.79 (0.22)	
P_{PR}(BKG)	2.51 (0.02)		2.56 (0.02)		2.24 (0.03)		**2.57** (0)		2.56 (0.01)	
Iteration	6.3 (1.24)		10 (4.63)		16 (0)		13.8 (1.64)		13.8 (2.16)	

D. Houston Data

The fourth image scene used for experiment was Houston dataset. Fig. 10 shows the hard classification maps produced by all test methods. However, it would be difficult to assess the effectiveness of the test methods if we simply compare their results in Fig. 10(b)–(f) with the GT in Fig. 10(a) by visual inspection. This is because the classes are too small with complicated BKG. Table XIII summarizes their averaged P_A, P_{OA}, P_{AA}, P_{APR}, and P_{PR}(BKG) values, and Table XIV summarizes the AUC measures from 3-D ROC analysis.

The conclusions drawn from Houston were very similar to that from University of Pavia. If we only used the results in Table XIII produced by hard classification measures, $P_{OA}/P_{AA}/P_{APR}$, IRTS-EPF-G-g, IRTS-NKCEM, and IRTS-NKLCMV performed very closely. On the contrary, if we use soft classification measures in Table XIV to assess the effectiveness of all test methods, IRTS-NKCEM was the best one that produced the highest O/AAUC$_{(D,F)}$ and O/AAUC$_{SNPR}$. Interestingly, it was discovered that P_{PR}(BKG) of Houston data were much lower than that of other three image scenes. This demonstrates that the BKG issue of Houston data is, indeed, very serious and cannot be disregarded during classification.

E. Comparative Analysis Using Various Training Sample Sizes

Because the number of used training samples could have significant impact on classification performance, Tables XV–XVIII also summarize the results calculated by both hard and soft classification measures using various training sample sizes. The OScore and AScore defined in (48) and (49) were also used to evaluate overall performance of classifiers and P_{PR}/P_{APR} was calculated by including BKG samples. The ICEM results were also included for comparison.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

TABLE XIV

AVERAGED $AUC_{(D,F)}$, $AUC_{(F,\tau)}$, $O/AAUC_{(D,F)}$ $O/AAUC_{(D,\tau)}$, $O/AAUC_{(F,\tau)}$, AND O/AAUCSNPR RESULTING FROM FIVE TIMES IEPF-G-g, IRTS-EPF-G-g, IRTS-CEM, IRTS-NKCEM, AND IRTS-NKLCMV FOR HOUSTON DATA

$n_{training}$	**IEPF-G-g** 500		**IRTS-EPF-G-g** 500		**IRTS-CEM** 500		**IRTS-NKCEM** 500		**IRTS-NKLCMV** 500	
class	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$	$AUC_{(D,F)}$	$AUC_{(F,\tau)}$
1	0.9879	0.0499	0.9897	0.0427	0.9615	0.0089	0.9952	0.0092	**0.9956**	**0.0079**
2	0.9841	0.1062	0.9877	0.0694	0.9735	0.0096	0.9950	0.0091	**0.9956**	**0.0090**
3	0.9999	0.0007	0.9999	**0.0005**	**1.0000**	0.0034	**1.0000**	0.0006	**1.0000**	0.0006
4	0.9751	0.1192	0.9801	0.0956	0.9321	0.0161	0.9910	0.0126	**0.9919**	**0.0124**
5	0.9979	0.0199	0.9982	0.0106	0.9980	0.0056	**0.9986**	0.0052	0.9985	**0.0051**
6	0.9994	0.0021	0.9962	**0.0008**	**0.9998**	0.0029	0.9968	0.0017	0.9994	0.0019
7	0.9648	0.0880	0.9768	0.0819	0.8888	0.0159	0.9871	**0.0138**	**0.9878**	0.0146
8	0.8968	0.1011	0.9449	0.2045	0.8675	0.0163	**0.9823**	0.0147	0.9767	**0.0140**
9	0.9401	0.1332	0.9588	0.1079	0.9198	0.0199	**0.9861**	**0.0139**	0.9852	0.0181
10	0.9885	0.0273	0.9896	0.0213	0.9610	0.0128	**0.9953**	**0.0095**	0.9952	0.0100
11	0.9623	0.0836	0.9729	0.0672	0.8903	**0.0103**	**0.9942**	0.0122	**0.9942**	0.0115
12	0.9746	0.0372	0.9869	0.0337	0.9075	**0.0073**	**0.9911**	0.0141	0.9890	0.0140
13	0.9767	0.0631	0.9885	0.0522	0.9851	**0.0050**	0.9944	0.0086	**0.9961**	0.0119
14	0.9999	0.0016	**1.0000**	0.0007	**1.0000**	0.0020	**1.0000**	0.0007	**1.0000**	**0.0005**
15	0.9998	0.0017	**0.9999**	0.0011	**0.9999**	0.0045	**0.9999**	0.0013	**0.9999**	**0.0012**
$O/AAUC_{(D,F)}$	0.9719/0.9765		0.9817/0.9847		0.9411/0.9523		**0.9928/0.9938**		0.9924/0.9937	
$O/AAUC_{(D,\tau)}$	0.7816/0.7862		**0.8004/0.8034**		0.3302/0.3848		0.4742/0.5293		0.4712/0.5276	
$O/AAUC_{(F,\tau)}$	0.0660/0.0557		0.0624/0.0527		0.0108/0.0089		**0.0099/0.0085**		0.0102/0.0088	
$O/AAUC_{SNPR}$	11.84/14.12		12.84/15.29		30.56/41.06		**48.16/62.77**		46.18/59.94	

TABLE XV

AVERAGED SOFT AND BINARY CLASSIFICATION PERFORMANCE RESULTING FROM FIVE TIMES IEPF-G-g, IRTS-EPF-G-g, IRTS-CEM, IRTS-NKCEM, AND IRTS-NKLCMV USING THREE DIFFERENT TRAINING SAMPLE NUMBERS FOR INDIANA PINES DATA

$n_{training}$	Method	P_{OA}/P_{AA} (%)	P_{PR}/P_{APR} (%)	$O/AAUC_{(D,F)}$	$O/AAUC_{(D,\tau)}$	$O/AAUC_{(F,\tau)}$	$O/AAUC_{SNPR}$	O/AScore
100	IEPFG-g	73.62/82.64	35.89/51.68	0.9202/0.9538	0.6224/0.6885	0.0565/0.0400	11.42/17.18	2.5812/2.9455
	IRTS-EPFG-g	84.78/91.01	41.33/**57.78**	0.9577/0.9752	**0.6687/0.7221**	0.0487/0.0378	13.95/19.14	**2.8388/3.1474**
	IRTS-CEM	70.28/81.57	34.26/44.35	0.8591/0.9364	0.2423/0.3801	0.0625/0.0410	3.90/9.30	2.0843/2.5347
	IRTS-NKCEM	91.14/95.28	44.43/48.15	0.9707/0.9866	0.2200/0.3788	0.0165/0.0108	13.50/35.34	2.5299/2.7889
	IRTS-NKLCMV	**93.16/96.03**	**45.41**/48.35	**0.9765/0.9887**	0.2226/0.3694	**0.0144/0.0096**	**15.57/38.96**	2.5704/2.7923
500	IEPFG-g	94.84/96.83	46.23/54.90	0.9824/0.9878	0.7793/0.7863	0.0454/0.0340	17.18/23.16	3.1270/3.2574
	IRTS-EPFG-g	98.22/98.60	47.88/**57.45**	0.9899/0.9929	**0.7924/0.7989**	0.0455/0.0330	17.46/24.19	**3.1978/3.3193**
	IRTS-CEM	95.66/97.19	46.63/55.64	0.9830/0.9926	0.4420/0.5521	0.0494/0.0300	8.96/12.42	2.7985/3.0430
	IRTS-NKCEM	99.48/**99.71**	48.49/52.78	**0.9969/0.9985**	0.3870/0.5310	**0.0105/0.0065**	**37.20/82.98**	2.8531/3.0479
	IRTS-NKLCMV	**99.50/99.71**	**48.50**/53.23	0.9966/0.9984	0.4224/0.5432	0.0143/0.0082	30.97/69.58	2.8848/3.0628
1000	IEPFG-g	97.91/98.50	47.73/57.64	0.9900/0.9933	**0.7978/0.7990**	0.0443/0.0329	18.03/24.27	**3.1999**/3.3208
	IRTS-EPFG-g	99.17/99.12	48.34/**57.72**	0.9902/0.9932	0.7771/0.7973	0.0443/0.0324	17.53/24.60	3.1981/**3.3265**
	IRTS-CEM	98.26/98.87	47.90/57.27	0.9952/0.9979	0.5155/0.6013	0.0414/0.0250	12.48/24.11	2.9309/3.1356
	IRTS-NKCEM	**99.80/99.89**	**48.65**/53.46	**0.9980/0.9991**	0.4898/0.6042	**0.0111/0.0065**	44.87/93.87	2.9612/3.1303
	IRTS-NKLCMV	99.73/99.84	48.61/53.04	0.9979/0.9990	0.4881/0.6022	**0.0111**/0.0066	**45.29/94.97**	2.9583/3.1234
GT	ICEM	99.24/99.24	48.38/42.50	0.9986/0.9992	0.6695/0.6942	0.0213/0.0146	31.39/47.58	3.1230/3.0962

TABLE XVI

AVERAGED SOFT AND BINARY CLASSIFICATION PERFORMANCE RESULTING FROM FIVE TIMES IEPF-G-g, IRTS-EPF-G-g, IRTS-CEM, IRTS-NKCEM, AND IRTS-NKLCMV USING THREE DIFFERENT TRAINING SAMPLE NUMBERS FOR SALINAS DATA

$n_{training}$	Method	OA/AA	P_{PR}/P_{APR}	$O/AAUC_{(D,F)}$	$O/AAUC_{(D,\tau)}$	$O/AAUC_{(F,\tau)}$	$O/AAUC_{SNPR}$	O/AScore
100	IEPFG-g	86.97/91.08	42.37/61.58	0.9624/0.9789	0.7754/0.8035	0.0509/0.0361	17.01/22.30	2.9803/3.2729
	IRTS-EPFG-g	**95.24/96.17**	**46.40/61.77**	0.9881/0.9883	**0.8331/0.8305**	0.0341/0.0332	27.00/25.09	**3.2035/3.3650**
	IRTS-CEM	84.25/87.22	41.05/56.80	0.9328/0.9687	0.2867/0.3338	0.0468/0.0320	6.16/9.34	2.4250/2.7107
	IRTS-NKCEM	93.01/97.28	45.31/54.31	**0.9920/0.9963**	0.2977/0.3760	**0.0117/0.0092**	**27.89/44.15**	2.6612/2.8790
	IRTS-NKLCMV	93.26/97.13	45.53/52.35	0.9916/0.9960	0.2928/0.3583	0.0120/0.0093	26.94/42.76/	2.6603/2.8398
500	IEPFG-g	97.92/98.61	47.71/59.35	0.9879/0.9878	0.8751/0.8764	0.0362/0.0332	25.46/26.38	3.2831/**3.4106**
	IRTS-EPFG-g	98.58/99.04	48.03/60.22	0.9881/0.8797	**0.8791/0.8793**	0.0372/0.0311	24.88/26.53	**3.2961**/3.3205
	IRTS-CEM	96.94/97.86	47.23/**64.73**	0.9914/0.9964	0.4640/0.5102	0.0352/0.0257	13.19/19.83	2.8619/3.1068
	IRTS-NKCEM	99.15/99.65	48.31/59.26	0.9978/0.9988	0.4502/0.5277	0.0068/**0.0049**	69.14/**110.22**	2.9158/3.1107
	IRTS-NKLCMV	**99.35/99.72**	**48.40**/58.75	**0.9983/0.9990**	0.4473/0.5325	**0.0064**/0.0050	**69.71**/107.50	2.9167/3.1112
1000	IEPFG-g	98.85/99.15	48.01/58.69	0.9892/0.9882	0.8850/0.8820	0.0314/0.0328	28.24/26.88	3.3114/3.4158
	IRTS-EPFG-g	98.98/99.33	48.22/59.79	0.9885/0.9878	0.8871/0.8846	0.0322/0.0327	27.63/27.04	**3.3154/3.4309**
	IRTS-CEM	98.94/99.15	48.21/**65.07**	0.9982/0.9989	0.5457/0.5781	0.0297/0.0216	18.37/26.83	2.9857/3.1976
	IRTS-NKCEM	99.44/99.77	48.45/59.54	**0.9989/0.9993**	**0.5249/0.6009**	0.0071/0.0052	74.69/115.90	2.9956/3.1881
	IRTS-NKLCMV	**99.69/99.88**	**48.57**/59.68	**0.9989/0.9993**	0.5265/0.6036	**0.0065/0.0050**	**82.33/121.79**	3.0015/3.1935
GT	ICEM	99.89/99.87	48.67/46.05	0.9998/0.9998	0.7529/0.7201	0.0090/0.0080	83.20/90.42	3.2293/3.1711

As summarized in Table XVIII, IRTS-NKCEM/NKLCMV always produced the highest $P_{OA}/P_{AA}/P_{PR}$ among all the test methods no matter how many training samples were used. On the contrary, the best classification performance measured by soft classification measures as soft classification performance was always produced by IRTS-NKCEM/NKLCMV

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

TABLE XVII

AVERAGED SOFT AND BINARY CLASSIFICATION PERFORMANCE RESULTING FROM FIVE TIMES IEPF-G-g, IRTS-EPF-G-g, IRTS-CEM, IRTS-NKCEM, AND IRTS-NKLCMV USING THREE DIFFERENT TRAINING SAMPLE NUMBERS FOR UNIVERSITY OF PAVIA DATA

n_{training}	Method	P_{OA}/P_{AA}	P_{PR}/P_{APR}	$O/AAUC_{(D,F)}$	$O/AAUC_{(D,\tau)}$	$O/AAUC_{(F,\tau)}$	$O/AAUC_{SNPR}$	O/AScore
100	IEPFG-g	84.02/91.49	17.33/19.02	0.9055/0.9495	0.6526/0.7046	0.1168/0.0791	5.67/5.93	2.4548/2.6801
	IRTS-EPFG-g	**95.84**/**97.26**	**19.77**/24.48	0.9461/0.9695	**0.6984**/**0.7491**	0.1232/0.0748	5.69/10.02	**2.6774**/**2.8612**
	IRTS-CEM	73.88/77.98	15.24/**25.53**	0.8099/0.8940	0.1879/0.2781	0.0570/0.0399	3.35/7.10	1.8320/2.1673
	IRTS-NKCEM	94.06/94.69	19.40/24.72	0.9514/0.9756	0.1433/0.2341	0.0177/0.0140	8.26/16.94	2.2116/2.3898
	IRTS-NKLCMV	93.77/95.50	19.34/23.31	**0.9550**/**0.9784**	0.1337/0.2245	**0.0131**/**0.0105**	10.27/21.43	2.2067/2.3805
500	IEPFG-g	96.76/97.97	19.96/23.14	0.9635/0.9747	0.7454/0.7429	0.1140/0.0768	6.57/9.67	2.7621/2.8519
	IRTS-EPFG-g	99.09/99.02	20.44/23.49	0.9731/0.9792	**0.7600**/**0.7534**	0.1134/0.0762	6.71/9.89	**2.8150**/**2.8815**
	IRTS-CEM	90.92/89.55	18.61/**28.69**	0.9272/0.9570	0.3002/0.3789	0.0401/0.0268	7.52/14.17	2.2826/2.4915
	IRTS-NKCEM	99.09/98.86	20.44/26.86	0.9845/0.9910	0.2419/0.3506	0.0148/0.0106	16.30/33.32	2.4069/2.5882
	IRTS-NKLCMV	**99.14**/**98.99**	**20.45**/26.56	**0.9847**/**0.9914**	0.2401/0.3671	**0.0134**/**0.0104**	**17.88**/**35.68**	2.4073/2.6036
1000	IEPFG-g	98.34/98.82	20.28/23.01	0.9685/0.9765	0.7521/0.7522	0.1103/0.0761	6.88/9.89	2.7965/2.8709
	IRTS-EPFG-g	**99.48**/**99.57**	20.52/25.37	0.9688/0.9767	**0.7682**/**0.7645**	0.1132/0.0783	6.82/9.78	**2.8328**/**2.9123**
	IRTS-CEM	91.30/91.50	18.83/24.38	0.9479/0.9607	0.3614/0.4360	0.0415/0.0303	8.88/14.62	2.3691/2.5252
	IRTS-NKCEM	99.39/99.08	20.50/27.38	**0.9887**/**0.9934**	0.2978/0.4194	**0.0137**/**0.0102**	**22.05**/**41.08**	2.4717/2.6672
	IRTS-NKLCMV	99.39/99.06	**20.50**/**27.39**	0.9887/0.9932	0.2885/0.4082	0.0146/0.0109	19.81/37.69	2.4615/2.6550
GT	ICEM	95.73/95.28	19.74/24.46	0.9827/0.9822	0.4313/0.4538	0.0196/0.0168	22.01/27.03	2.5491/2.6166

TABLE XVIII

AVERAGED SOFT AND BINARY CLASSIFICATION PERFORMANCE RESULTING FROM FIVE TIMES IEPF-G-g, IRTS-EPF-G-g, IRTS-CEM, IRTS-NKCEM, AND IRTS-NKLCMV USING THREE DIFFERENT TRAINING SAMPLE NUMBERS FOR HOUSTON DATA

n_{training}	Method	P_{OA}/P_{AA}	P_{PR}/P_{APR}	$O/AAUC_{(D,F)}$	$O/AAUC_{(D,\tau)}$	$O/AAUC_{(F,\tau)}$	$O/AAUC_{SNPR}$	O/AScore
150	IEPFG-g	78.08/80.99	2.03/13.48	0.8898/0.9045	0.6345/0.6580	0.0563/0.0532	11.38/12.38	2.2691/2.4540
	IRTS-EPFG-g	86.91/88.31	2.26/22.36	0.9420/0.9501	**0.6995**/**0.7131**	0.0555/0.0487	12.73/14.65	**2.4777**/**2.7212**
	IRTS-CEM	75.14/77.30	1.95/22.32	0.8999/0.9188	0.2587/0.3050	0.0183/0.0138	14.25/19.54	1.9113/2.2043
	IRTS-NKCEM	**95.03**/**95.37**	**2.47**/**17.47**	**0.9792**/**0.9826**	0.3653/0.4330	**0.0074**/**0.0065**	**49.27**/**66.58**	2.3121/2.5375
	IRTS-NKLCMV	94.44/95.04	2.45/17.71	0.9769/0.9802	0.3554/0.4217	0.0081/0.0072	43.92/59.20	2.2930/2.5223
750	IEPFG-g	96.78/97.11	2.51/13.58	0.9719/0.9765	0.7816/0.7862	0.0660/0.0557	11.84/14.12	2.6805/2.8140
	IRTS-EPFG-g	98.50/**98.63**	2.56/19.87	0.9817/0.9847	**0.8004**/**0.8043**	0.0624/0.0527	12.84/15.29	**2.7303**/**2.9213**
	IRTS-CEM	86.24/87.73	2.24/**20.06**	0.9411/0.9523	0.3302/0.3848	0.0108/0.0094	30.55/41.06	2.1453/2.4056
	IRTS-NKCEM	**98.79**/98.50	**2.57**/18.01	**0.9928**/**0.9938**	0.4742/0.5293	**0.0099**/0.0085	**48.16**/**62.78**	2.4707/2.6797
	IRTS-NKLCMV	98.59/98.33	2.56/17.96	0.9924/0.9937	0.4712/0.5276	0.0102/0.0088	46.18/59.94	2.4648/2.6753
1200	IEPFG-g	97.93/98.05	2.54/15.22	0.9761/0.9800	0.7858/0.7895	0.0656/0.0551	11.97/14.34	2.7009/2.8470
	IRTS-EPFG-g	**99.44**/**99.41**	**2.58**/15.97	0.9883/0.9860	0.8027/0.8051	0.0663/0.0555	12.10/14.50	**2.7442**/**2.8898**
	IRTS-CEM	88.87/90.32	2.31/**17.61**	0.9479/0.9581	0.3329/0.3873	**0.0090**/**0.0078**	37.24/49.84	2.1836/2.4169
	IRTS-NKCEM	99.05/98.81	2.57/17.59	**0.9937**/**0.9947**	**0.5194**/**0.5666**	0.0098/0.0084	**53.05**/67.79	2.5196/2.7169
	IRTS-NKLCMV	98.83/98.72	2.57/17.48	0.9935/**0.9947**	0.5028/0.5527	0.0102/0.0087	49.54/**69.73**	2.5001/2.7007
GT	ICEM	89.72/91.27	2.33/12.22	0.9559/0.9645	0.3524/0.4064	0.0069/0.0060	50.92/68.02	2.2219/2.3998

with the highest $O/AAUC_{(D,F)}$ and $O/AAUC_{SNPR}$. On some occasions, IRTS-EPF-G-g might be able to produce little better of P_{OA}/P_{AA} than IRTS-NKCEM and IRTS-NKLCMV. However, the classification performance of IRTS-EPF-G-g measured by soft classification measures such as $O/AAUC_{(D,F)}$ and $O/AAUC_{SNPR}$ was still worse than IRTS-NKCEM/NKLCMV, which was mainly because P_{OA} and P_{AA} were unable to evaluate the BKG suppressibility of classifiers. Furthermore, when n_{training} was large enough (i.e., $n_{\text{training}} = 1000$), IRTS-NKCEM/NKLCMV could even achieve better classification performance with higher P_{OA}, P_{AA}, P_{PR}, and $O/AAUC_{SNPR}$ than ICEM. In addition, the results also showed that IRTS-CEM was as effective as IRTS-NKCEM, IRTS-NKLCMV, and IRTS-EPF-G-g because the latter was nonlinear classifiers compared to the former, ICEM, which was a linear classifier.

As for the Houston experiments, when 1200 training samples were used, IRTS-EPF-G-g outperformed IRTS-NKCEM/NKLCMV in terms of both P_{OA} and P_{AA}. However, once we reduced the size of training samples to 150 (nearly 1% of the GT), IRTS-NKCEM/NKLCMV improved almost 9% of P_{OA} over IRTS-EPF-G-g. This was because EPF used as a pure pixel-based classifier was not able to deal with BKG effects when only few training samples were used. In this case, IRTS-NKCEM/NKLCMV utilized $\mathbf{K}_{N\times N}^{-1}$ to suppress the interference of BKG. As a result, IRTS-NKCEM/NKLCMV performed better than IRTS-EPF-G-g. This is particularly important because the available training samples are usually very limited in practical applications. Once again, IRTS-NKCEM/NKLCMV significantly improved IRTS-CEM and ICEM with the highest $O/AAUC_{(D,F)}$ and $O/AAUC_{SNPR}$.

Finally, it should be noted that two EPF-based classifiers produced the highest O/AScore in Tables XV–XVIII. This because EPF produced high $O/AAUC_{(D,\tau)}$ values at the expense of $O/AAUC_{(F,\tau)}$ values. Such scenario is commonly found in target detection where the AUC of 2-D ROC curve of P_D versus P_F, $AUC_{(D,F)}$ values were very high, so were $AUC_{(F,\tau)}$ values [39].

F. Time Complexity and Analysis of Test Algorithms

The Nyström method was proposed to address the high computational complexity resulting from manipulating $\mathbf{K}_{N\times N}$. In general, KCEM has problem with processing a large dataset because it must kernelize each data sample. For instance, Salinas has 111104 of data samples, which requires 98 GB of memory to store $\mathbf{K}_{N\times N}$. The NKCEM provides a feasible solution by replacing $\mathbf{K}_{N\times N}$ with $\mathbf{E}_{m_N\times N}$, where the

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

TABLE XIX
RUNNING TIME (S) OF NKCEM/NKLCMV USING DIFFERENT mN/N VALUES FOR INDIAN PINES

	Original	Nyström method		
		$m_N/N = 0.5$	$m_N/N = 0.25$	$m_N/N = 0.1$
KCEM	874.91	316.44	87.78	16.50
KLCMV	506.47	102.98	31.66	7.42

TABLE XX
COMPUTING TIME (S) OF ALL TEST METHODS FOR FOUR IMAGE SCENES

	IP	Salinas	U. Pavia	Houston
IEPF-G-g	14.86	38.16	20.96	197.85
IRTS-EPFG-g	22.87	57.83	28.18	403.7
IRTS-CEM	1.460	6.53	5.15	56.99
IRTS-NKCEM	406.1	822.4	1078.26	3432.1
IRTS-NKLCMV	173.7	634.96	720.49	1932.9

TABLE XXI
AVERAGE P_{OA}/P_{AA} RESULTING FROM FOUR TEST METHODS

	IP	Salinas	U. Pavia	Houston
IID	90.74/93.62	98.92/99.32	98.87/98.99	95.10/95.36
OTVCA	89.27/93.33	98.40/98.98	95.83/95.99	93.74/94.74
IRTS-NKCEM	99.48/**99.71**	99.15/99.65	99.09/98.86	**98.79**/**98.50**
IRTS-NKLCMV	**99.50**/**99.71**	**99.35**/**99.72**	**99.14**/**98.99**	98.59/98.33

TABLE XXII
AVERAGE $P_{PR}(BKG)/P_{APR}(BKG)$ RESULTING FROM FOUR TEST METHODS

	IP	Salinas	U. Pavia	Houston
IID	44.23/47.68	48.20/**60.95**	20.39/18.47	2.47/13.55
OTVCA	43.51/38.61	47.94/53.85	19.77/22.47	2.44/12.33
IRTS-NKCEM	48.49/52.78	48.31/59.26	20.44/**26.86**	**2.57**/**18.01**
IRTS-NKLCMV	**48.50**/**53.23**	**48.40**/58.75	**20.45**/26.56	2.56/17.69

memory and time complexity can be significantly reduced. As a consequence, the memory complexity of computing $\mathbf{K}_{N\times N}^{-1}$ in (17) is reduced from $O(N^2)$ to $O(m_N^2)$ by the Nyström method. Table XIX summarizes the running times of NKCEM/NKLCMV using different Nyström sampling rates, m_N/N, for Indian Pines, where the original KCEM/KLCMV was the least efficient compared to NKCEM/NKLCMV and the computer environment used for experiments was 2.90-GHz Intel core i7 CPU and 32 GB of memory.

Table XX also summarizes running times for all test methods where it is clear to show that IRTS-CEM was the most efficient because it is linear and two EPF-based methods used SVM as spectral classifier that required kernelizing each training sample, which resulted in running time longer than CEM. On the contrary, NKCEM/NKLCMV was the slowest because they must kernelize all data samples.

Interestingly, despite that the classification performances of IRTS-NKCEM and IRTS-NKLCMV were close, IRTS-NKLCMV was faster than IRTS-NKCEM because NKLCMV classified all classes in one-shot operation compared to NKCEM, which classified one class at a time.

G. Comparing With State-of-the-Art SS methods

In this section, two state-of-the-art SS methods were compared to IRTS-NKCEM and IRTS-NKLCMV, which are OTVCA [50], [51], and feature extraction based on IID [52]. However, we would like to point out that the two state-of-the-art methods are pure pixel-based classifiers that only produced binary classification results rather the real-valued class abundance fractional maps, in which case the 3-D ROC analysis is not applicable to evaluation. Therefore, only hard classification measures, P_{OA} and P_{PR} (BKG), will be used. The numbers of training samples used for each image scene are summarized at Table VI.

Tables XXI and XXII summarizes the classification measure of four test methods. It was clear that two proposed classifiers, IRTS-NKCEM/NKLCMV, always outperformed OTVCA and IID with the highest P_{OA} and $P_{PR}(BKG)$. There is a major difference between IRTS-NKCEM/NKLCMV and two compared methods where the former utilizes Gaussian-filtered feedback process to capture spatial information as opposed to the latter, which relies on solving optimization problem to extract SS features from the original data cube for classification. In the latter case, the *a priori* spatial information captured from the original data cube is very likely to be affected by noise or BKG. As summarized in Tables XXI and XXII, even though IID and OTVCA were able to perform well for Salinas, their classification performances on Indian Pines and Houston dataset were not.

XI. CONCLUSION

This article develops two kernel mixed pixel classification methods, IRTS-NKCEM and IRTS-NKLCMV, for HMPC, which extend the linear classifiers, ICEM [27], IRTS-CEM [32], and ILCMV [30], [31], into their kernel counterparts to address nonlinear class inseparability issues that cannot be solved by linear classifiers. because HMPC generally produces real-valued class abundance fractional maps, the traditional classification measures, kappa/AA/OA, may not work effectively because they are designed based on discrete classification maps as hard classification measures. To address this issue, 3-D ROC analysis derived from detection theory is also used as an additional evaluation tool to evaluate performance of mixed pixel classifiers where several new 3-D ROC curve-based detection measures in [39], [40] are also developed as soft classification measures. The experimental results based on the four benchmark datasets demonstrate that IRTS-NKCEM/NKLCMV not only produce the best classification performance but also turn out to be the most effective classifiers in terms of 3-D ROC curved-derived soft classification measures. In particular, the experimental results of Houston and University of Pavia clearly show that IRTS-NKCEM/NKLCMV are able to use very small and limited sets of training samples to produce better classification results than ICEM using complete GT as well as other SS methods. In addition, IRTS-NKCEM/NKLCMV can improve IRTS-CEM and ICEM and can be also shown to outperform

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

state-of-the-art SS methods at the expense of computational cost, which requires longer computing time than other test classifiers even though the two algorithms have already been sped up by the Nyström method. So, in future, to better perform using the Nyström method, selecting appropriate LRA sampling rate m_N/N still needs to be further investigated.

REFERENCES

[1] F. Melgani and L. Bruzzone, "Classification of hyperspectral remote sensing images with support vector machines," *IEEE Trans. Geosci. Remote Sens.*, vol. 42, no. 8, pp. 1778–1790, Aug. 2004.

[2] Y. Chen, L. Zhu, P. Ghamisi, X. Jia, G. Li, and L. Tang, "Hyperspectral images classification with Gabor filtering and convolutional neural network," *IEEE Geosci. Remote Sens. Lett.*, vol. 14, no. 12, pp. 2355–2359, Dec. 2017.

[3] J. A. Benediktsson, J. A. Palmason, and J. R. Sveinsson, "Classification of hyperspectral data from urban areas based on extended morphological profiles," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 3, pp. 480–491, Mar. 2005.

[4] X. Kang, S. Li, and J. A. Benediktsson, "Spectral–spatial hyperspectral image classification with edge-preserving filtering," *IEEE Trans. Geosci. Remote Sens.*, vol. 52, no. 5, pp. 2666–2677, May 2014.

[5] G. Vamps-Valls and L. Bruzzone, "Kernel-based methods for hyperspectral image classiication," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 6, pp. 1351–1362, Jun. 2005.

[6] G. Camps-Valls, L. Gomez-Chova, J. Munoz-Mari, J. Vila-Frances, and J. Calpe-Maravilla, "Composite kernels for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 3, no. 1, pp. 93–97, Jan. 2006.

[7] J. Li, J. M. Bioucas-Dias, and A. Plaza, "Spectral–spatial classification of hyperspectral data using loopy belief propagation and active learning," *IEEE Trans. Geosci. Remote Sens.*, vol. 51, no. 2, pp. 844–856, Feb. 2013.

[8] J. Peng, Y. Zhou, and C. L. P. Chen, "Region-kernel-based support vector machines for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 9, pp. 4810–4824, Sep. 2015.

[9] Y. Tarabalka, J. A. Benediktsson, and J. Chanussot, "Spectral–spatial classification of hyperspectral imagery based on partitional clustering techniques," *IEEE Trans. Geosci. Remote Sens.*, vol. 47, no. 8, pp. 2973–2987, Aug. 2009.

[10] Y. Tarabalka, M. Fauvel, J. Chanussot, and J. A. Benediktsson, "SVM- and MRF-based method for accurate classification of hyperspectral images," *IEEE Geosci. Remote Sens. Lett.*, vol. 7, no. 4, pp. 736–740, Oct. 2010.

[11] M. Fauvel, Y. Tarabalka, J. A. Benediktsson, J. Chanussot, and J. C. Tilton, "Advances in spectral–spatial classification of hyperspectral images," *Proc. IEEE*, vol. 101, no. 3, pp. 652–675, Mar. 2013.

[12] P. Ghamisi, M. D. Mura, and J. A. Benediktsson, "A survey on spectral–spatial classification techniques based on attribute profiles," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 5, pp. 2335–2353, May 2015.

[13] T. Lu, S. Li, L. Fang, L. Bruzzone, and J. A. Benediktsson, "Set-to-set distance-based spectral–spatial classification of hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 12, pp. 7122–7134, Dec. 2016.

[14] Z. Zhong, J. Li, Z. Luo, and M. Chapman, "Spectral-spatial residual network for hyperspectral image classification: A 3D deep learning framework," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 2, pp. 847–858, Feb. 2018.

[15] P. Ghamisi *et al.*, "New frontiers in spectral–spatial hyperspectral image classification," *IEEE Geosci. Remote Sens. Mag.*, vol. 6, no. 3, pp. 10–43, Sep. 2018.

[16] F. Cao *et al.*, "Sparse representation-based augmented multinomial logistic extreme learning machine with weighted composite features for spectral–spatial classification of hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 11, pp. 6263–6279, Nov. 2018.

[17] J. M. Haut, M. E. Paoletti, J. Plaza, J. Li, and A. Plaza, "Active learning with convolutional neural networks for hyperspectral image classification using a new Bayesian approach," *IEEE Trans. Geosci. Remote Sens.*, vol. 56, no. 11, pp. 6440–6461, Nov. 2018.

[18] S. Zhong, C.-L. Chang, and Y. Zhang, "Iterative edge preserving filtering approach to hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, vol. 16, no. 1, pp. 90–94, Jan. 2019.

[19] S. Zhong, Y. Zhang, and C.-I Chang, "A spectral–spatial feedback close network system for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 12, pp. 10056–10069, Dec. 2019.

[20] C.-I Chang, K. Y. Ma, C.-C. Liang, Y.-M. Kuo, S. Chen, and S. Zhong, "Iterative random training sampling spectral spatial classification for hyperspectral images," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 13, pp. 3986–4007, Jul. 2020.

[21] P. Duan, X. Kang, S. Li, P. Ghamisi, and J. A. Benediktsson, "Fusion of multiple edge-preserving operations for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 12, pp. 10336–10349, Dec. 2019.

[22] P. Duan, P. Ghamisi, X. Kang, B. Rasti, S. Li, and R. Gloaguen, "Fusion of dual spatial information for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, early access, Nov. 11, 2020, doi: 10.1109/TGRS.2020.3031928.

[23] S. Zhong, S. Chen, C.-I Chang, and Y. Zhang, "Fusion of spectral–spatial classifiers for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 59, no. 6, pp. 5008–5027, Jun. 2021.

[24] J.C. Harsanyi, "Detection and classification of subpixel spectral signatures in hyperspectral image sequences," Ph.D. dissertation, Dept. Elect. Eng., Univ. Maryland, Baltimore County, MD, USA, Aug. 1993.

[25] W. Farrand, "Mapping the distribution of mine tailings in the Coeur d'Alene River Valley, Idaho, through the use of a constrained energy minimization technique," *Remote Sens. Environ.*, vol. 59, no. 1, pp. 64–76, Jan. 1997.

[26] C.-I Chang, *Hyperspectral Imaging: Techniques for Spectral detection and Classification*. Norwell, MA, USA: Kluwer, 2003.

[27] B. Xue *et al.*, "A subpixel target approach to hyperpsectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 55, no. 9, pp. 5093–5114, Sep. 2017.

[28] C.-I Chang, "Target signature-constrained mixed pixel classification for hyperspectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 40, no. 5, pp. 1065–1081, May 2002.

[29] O. L. Frost, "An algorithm for linearly constrained adaptive array processing," *Proc. IEEE*, vol. 60, no. 8, pp. 926–935, Aug. 1972.

[30] C. Yu, B. Xue, M. Song, Y. Wang, S. Li, and C.-I Chang, "Iterative target-constrained interference-minimized classifier for hyperspectral classification," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 11, no. 4, pp. 1095–1117, Apr. 2018.

[31] C. Yu, Y. Wang, M. Song, and C.-I Chang, "Class signature-constrained background-suppressed approach to band selection for classification of hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 1, pp. 14–31, Jan. 2019.

[32] X. Shang, M. Song, and C.-I Chang, "An iterative random training sample selection approach to constrained energy minimization for hyperspectral image classification," *IEEE Geosci. Remote Sens. Lett.*, early access, Jul. 2, 2020, doi: 10.1109/LGRS.2020.3005078.

[33] C. K. I. Williams and M. Seeger, "Using the Nyström method to speed up kernel machines," in *Proc. Neural Inf. Process. Syst. (NIPS)*, 2001, pp. 682–688.

[34] P. Drineas and M. W. Mahoney, "On the Nyström method for approximating a Gram matrix for improved kernel-based learning," *J. Mach. Learn. Res.*, vol. 6, pp. 2153–2175, Dec. 2005.

[35] A. Talwalkar, S. Kumar, and H. Rowley, "Large-scale manifold learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Anchorage, AK, USA, Jun. 2008, pp. 1–8.

[36] S. Kumar, M. Mohri, and A. Talwalkar, "On sampling-based approximate spectral decomposition," in *Proc. 26th Annu. Int. Conf. Mach. Learn. (ICML)*, 2009, pp. 585–592.

[37] K. Zhang and J. T. Kwok, "Clustered Nyström method for large scale manifold learning and dimension reduction," *IEEE Trans. Neural Netw.*, vol. 21, no. 10, pp. 1576–1587, Oct. 2010.

[38] J. A. P. Hidalgo, A. Perez-Suay, F. Nar, and G. Camps-Valls, "Efficient nonlinear RX anomaly detectors," *IEEE Geosci. Remote Sens. Lett.*, vol. 18, no. 2, pp. 231–235, Feb. 2021.

[39] C.-I Chang, "Multiparameter receiver operating characteristic analysis for signal detection and classification," *IEEE Sensors J.*, vol. 10, no. 3, pp. 423–442, Mar. 2010.

[40] C.-I Chang, "An effective evaluation tool for hyperspectral target detection: 3D receiver operating characteristic curve analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 11, pp. 8093–8115, Nov. 2020.

[41] M. Song, X. Shang, and C.-I Chang, "3-D receiver operating characteristic analysis for hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 58, no. 11, pp. 8093–8115, Nov. 2020.

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.

[42] C.-I Chang, "Statistical detection theory approach to hyperspectral image classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 57, no. 4, pp. 2057–2074, Apr. 2019.

[43] X. Jiao and C.-I Chang, "Kernel-based constrained energy minimization," *Proc. SPIE*, vol. 6966, Jun. 2008, Art. no. 69661S.

[44] B. Schölkopf, A. J. Smola, and K.-R. Müller, "Kernel principal component analysis," *Neural Comput.*, vol. 10, no. 5, pp. 1299–1319, 1998.

[45] H. Kwon and N. M. Nasrabadi, "Kernel RX-algorithm: A nonlinear anomaly detector for hyperspectral imagery," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 2, pp. 388–397, Feb. 2005.

[46] H. Kwon and N. M. Nasrabadi, "Kernel orthogonal subspace projection for hyperspectral signal classification," *IEEE Trans. Geosci. Remote Sens.*, vol. 43, no. 12, pp. 2952–2962, Dec. 2005.

[47] G. Baudat and F. Anouar, "Generalized discriminant analysis using a kernel approach," *Neural Comput.*, vol. 12, no. 10, pp. 2385–2404, Oct. 2000.

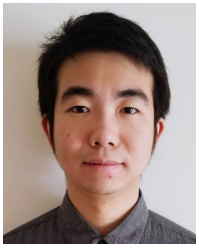
[48] T. Wang, B. Du, and L. Zhang, "A kernel-based target-constrained interference-minimized filter for hyperspectral sub-pixel target detection," *IEEE J. Sel. Topics Appl. Earth Observ. Remote Sens.*, vol. 6, no. 2, pp. 626–637, Apr. 2013.

[49] S. Theodoridis and K. Koutroumbas, *Pattern Recognition*. New York, NY, USA: Academic, 1999, p. 366.

[50] B. Rasti, M. O. Ulfarsson, and J. R. Sveinsson, "Hyperspectral feature extraction using total variation component analysis," *IEEE Trans. Geosci. Remote Sens.*, vol. 54, no. 12, pp. 6976–6985, Dec. 2016.

[51] B. Rasti *et al.*, "Feature extraction for hyperspectral imagery: The evolution from shallow to deep: Overview and toolbox," *IEEE Geosci. Remote Sens. Mag.*, vol. 8, no. 4, pp. 60–88, Dec. 2020.

[52] X. Kang, S. Li, L. Fang, and J. A. Benediktsson, "Intrinsic image decomposition for feature extraction of hyperspectral images," *IEEE Trans. Geosci. Remote Sens.*, vol. 53, no. 4, pp. 2241–2253, Apr. 2015.



Kenneth Yeonkong Ma received the B.S. degree in applied material and optoelectronic engineering from National Chi Nan University, Nantou County, Taiwan, in 2013, and the M.S. degree in optoelectronic engineering from the National Chung Hsing University, Taichung City, Taiwan, in 2017. He is pursuing the Ph.D. degree in electrical engineering with the Remote Sensing Signal and Image Processing Laboratory, University of Maryland, Baltimore County (UMBC), Baltimore, MD, USA.

He is a System Engineer with Peraton Corporation, Herndon, VA, USA, on the sustaining engineering team for Tracking and Data Relay Satellite (TDRS). His research interests include hyperspectral image processing, pattern recognition, machine learning, and digital signal processing.



Chein-I. Chang (Life Fellow, IEEE) received the B.S. degree in mathematics from Soochow University, Taipei, Taiwan, in 1973, the M.S. degree in mathematics from the Institute of Mathematics, National Tsing Hua University, Hsinchu, Taiwan, in 1975, the M.A. degree in mathematics from the State University of New York at Stony Brook, Stony Brook, NY, USA, in 1977, the M.S. and M.S.E.E. degrees from the University of Illinois at Urbana–Champaign, Urbana, IL, USA, in 1982, and the Ph.D. degree in electrical engineering from the University of Maryland, College Park, MD, USA, in 1987.

He has been with the University of Maryland, Baltimore County (UMBC), Baltimore, MD, USA, since 1987, where he is a Professor with the Department of Computer Science and Electrical Engineering. He is holding a Chang Jiang Scholar Chair Professorship and has been the Director of the Center for Hyperspectral Imaging in Remote Sensing (CHIRS), Dalian Maritime University, Dalian, China, since 2016. In addition, he also has been a Chair Professor of National Chiao Tung University, Hsinchu, since 2019. He has authored over four books, including *Hyperspectral Imaging: Techniques for Spectral Detection and Classification* (Kluwer Academic, 2003), *Hyperspectral Data Processing: Algorithm Design and Analysis* (John Wiley & Sons, 2013), *Real Time Progressive Hyperspectral Image Processing*: *Endmember Finding and Anomaly Detection* (Springer, 2016), and *Recursive Hyperspectral Sample and Band Processing: Algorithm Architecture and Implementation* (Springer, 2017). In addition, he also edited two books, *Recent Advances in Hyperspectral Signal and Image Processing*, (John Wiley & Sons, 2006) and *Hyperspectral Data Exploitation: Theory and Applications* (John Wiley & Sons, 2007) and co-edited with A. Plaza, *High Performance Computing in Remote Sensing* (CRC Press, 2007). He has seven patents on hyperspectral image processing. His research interests include multispectral/hyperspectral image processing, automatic target recognition, and medical imaging.

Dr. Chang is a fellow of SPIE. He received the National Research Council Senior Research Associateship Award from 2002 to 2003, sponsored by the U.S. Army Soldier and Biological Chemical Command, Edgewood Chemical and Biological Center, Aberdeen Proving Ground, MD, USA. He was a Plenary Speaker for Society for Photo-Optical Instrumentation Engineers (SPIE) Optics + Applications, Remote Sensing Symposium in 2009. He was the Guest Editor of a Special Issue on Telemedicine and Applications for the *Journal of High Speed Networks* (April 2000) and a Co-Guest Editor of a Special Issue on Broadband Multimedia Sensor Networks in Healthcare Applications for the *Journal of High Speed Networks* (April 2007). He was also a Co-guest Editor of a Special Issue on High Performance Computing of Hyperspectral Imaging for the *International Journal of High Performance Computing Applications* (December 2007), a special issue on Signal Processing and System Design in Health Care Applications for the *EURASIP Journal on Advances in Signal Processing* (2009), a special issue on Multispectral, Hyperspectral, and Polarimetric Imaging Technology for the *Journal of Sensors* (2016), and a special issue on Hyperspectral Imaging and Applications for *Remote Sensing* (2018).

Authorized licensed use limited to: Hohai University Library. Downloaded on November 05,2026 at 11:09:36 UTC from IEEE Xplore. Restrictions apply.